# Graduation Project Report

# for

# A Virtual Queuing System for Bank

**Prepared by Seray SAYAN 2385680, Eylül DenizYILDIZ 2385748,**

**İrem ÇİLOĞLU 2385292 & Kamil Barış GÖKMEN 2453223**

**Middle EastTechnical University Northern Cyprus Campus**

**Computer Engineering**

**Supervised by İDİL CANDAN**

**17/06/2023**

Contents

# 1 Introduction

## 1.1. Purpose

This document's major goal is to provide demonstrations of the specifications for the project's Virtual Queuing System for the Bank. Both the proposed functional and non-functional requirements are presented in detail in the document. Additionally, this report utilizes ER diagrams and UML diagrams to represent all of the inputs and outputs from the software system. This project tries to avoid long bank queues in banks. Thanks to the application, people can easily take their queue online by downloading the application to their mobile devices without going to the bank, where they will make transactions; this enables social distance, which has become necessary since COVID, to be easily resolved. Moreover, the chance to make good use of the time spent waiting in a queue for bank transactions arises. In addition, this project lists all bank branches from the nearest to the farthest by making logical suggestions to its users. At the same time, instead of constantly following the sequence number, the application provides convenience by sending notifications to its users when the queue numbers are approaching.

Moreover, unlike other projects, this project will be a general queue management system rather than belonging to a specific business. This way, the developed project will be usable where queue management is required, such as in any government office, bank, or hospital. Although the project was developed as a bank queue management system, we aim to solve this problem by using Firebase as a queue system independent of the bank without using the bank's own database. Finally, for example, a similar project made by WAVETEC[1] works with a simple queue management algorithm. In contrast, the project described in the report aims to differentiate from other systems by making a priority queue system, considering features such as age and customer types. It also implements a dynamic priority feature. Due to this functionality, the waiting time in the queue does not exceed a certain amount of time, even though it is a priority application. In other words, this feature aims to save the low-priority customer from waiting too long.

## 1.2 Scope

The main goal of this project is to develop a queue management system that supports various priorities, including age, customer types, and dynamic priority, which can be adjusted based on the application area. Moreover, thanks to the queue algorithms used in the application, although our application is valid for banks, it can be used in all areas where queue algorithms should be used in the future.

1. Priority queue management, where priorities will be managed with different factors, including age, customer types, and dynamic priority.

2. Providing benefits in time management and showing the nearest bank branches in the region to help the time management of customers. Moreover, it can help with time issues with implementing the dynamic priority feature.

3. Offering the chance to use time efficiently because of taking queues and following queue numbers remotely.

## 1.3 Identification of constraints

1. Economic Constraints:

- Cost: The development, maintenance, and hosting costs of the application and the admin panel should be considered to ensure economic viability ensuring that the benefits derived from its use outweigh the associated costs.

2. Environmental Constraints:

- Energy Efficiency: The application should be designed to minimize energy consumption on mobile devices to reduce environmental impact.
- Paperless Operations: Encouraging digital transactions and reducing the need for printed tickets or receipts can contribute to environmental sustainability.

3. Social Constraints:

- Accessibility: The application and admin panel should be inclusive and accessible to both young and elderly users.
- User Experience: Providing a user-friendly interface and intuitive interactions will enhance user satisfaction and adoption of the application and admin panel.

5. Ethical Constraints:

- Fairness: The application should ensure fair treatment of customers in the queue management system, avoiding bias or discrimination.

7. Manufacturability Constraints:

- Scalability: The application and admin panel should be designed to handle a large number of users and adapt to increasing demand without compromising performance.
- Compatibility: The application should be compatible with various mobile devices, operating systems, and screen sizes to reach a wider audience.

8. Sustainability Constraints:

- Long-Term Support: The application and admin panel should have a plan for continuous improvement, bug fixes, and updates to ensure its sustainability and longevity.

## 1.4 Related Work

Before deciding the environment that we use to build our Android application, we did research on related works. As in the project we have done, these are the most relevant ones as they offer geolocation services to customers, follow their current queue numbers, have features such as waiting time notification and instant notification. Therefore, we decided to useWavetech, Qudini, and 2 meters as related work and did a comparison with our project. In Table 1, three selected projects and our project's some features are compared such as notification, waiting time, qr code, and geolocation service.

| WORK | Instant Notification | Know Your Wait Time | Feedback | QR Codes | Analyze | Geolocation Services |
|---|---|---|---|---|---|---|
| WAVETEC | Yes | Yes | Yes | Yes | Yes | Yes |
| QUDINI | Yes | Yes | Yes | Yes | Yes | Yes |
| 2 METERS | Yes | No | Yes | Yes | Yes | No |
| PROJECT | YES | YES | NO | NO | YES | YES |

*Table 1: Related work table*

1. Enterprise Virtual Queue Management System

Wavetec has created a comprehensive system that can address a range of queuing requirements, from simple queuing demands to networked multi-branch, multi-region enterprise solutions. Customers and tourists can use queue management systems to get in line by buying tickets via a self-service ticketing kiosk, the web, a mobile application, or an online appointment[1]. According to the Table 1, Wavetec is one of the successful applications in queue management due to its advantageous features such as sending immediate notification to its customers, showing how long its customers must wait, having feedback section to improve itself, having QR codes for easy access, analysing its performance and supporting geolocation serves to provide its customers more efficient experience.

2. The Leading Walk-in Virtual Queue System for Enterprise Brands

QUDINI is an enterprise customer experience management platform that enables our clients and brands' online bookings to be maximised and optimised. Upon arrival, customers can join a virtual line, and social distancing ensures a total experience from beginning to end[2].According to Table 1, QUADINI is one of the successful applications in queue management due to its advantageous features such as sending immediate notification to its customers, showing how long its customers must wait, having feedback section to improve itself, having QR codes for easy access, analysing its performance and supporting geolocation serves to provide its customers more efficient experience.

3. 2 meters

A virtual queueing system and an appointment management tool, 2Meters, are highly effective. Their innovative algorithm seamlessly combines virtual lines and appointments. As a result, your staff can work on more pertinent things. 2Meters can be set up to suit your company's needs. Once customised, it continues to be adaptable enough to overcome unforeseen difficulties like high peak demand or extended servicing durations[3]. According to Table 1, 2 meters is not very efficient in queue management compared to other applications. It provides immediate notification to its customers, a feedback section to improve itself, QR codes for easy access and it analyses its performance. Unfortunately, it does not include knowing how much time customers must wait and supporting geolocation services. Because of these, customers can not make use of their waiting time in the line, also they may spend more time by not joining the queue of the nearest store to them.

## 1.5 Product Overview

### 1.5.1 Product perspective

Our product's representation in a context diagram is shown in Figure 1 below. We used Google Firebase, Simulation and Web Admin Panel as external systems.

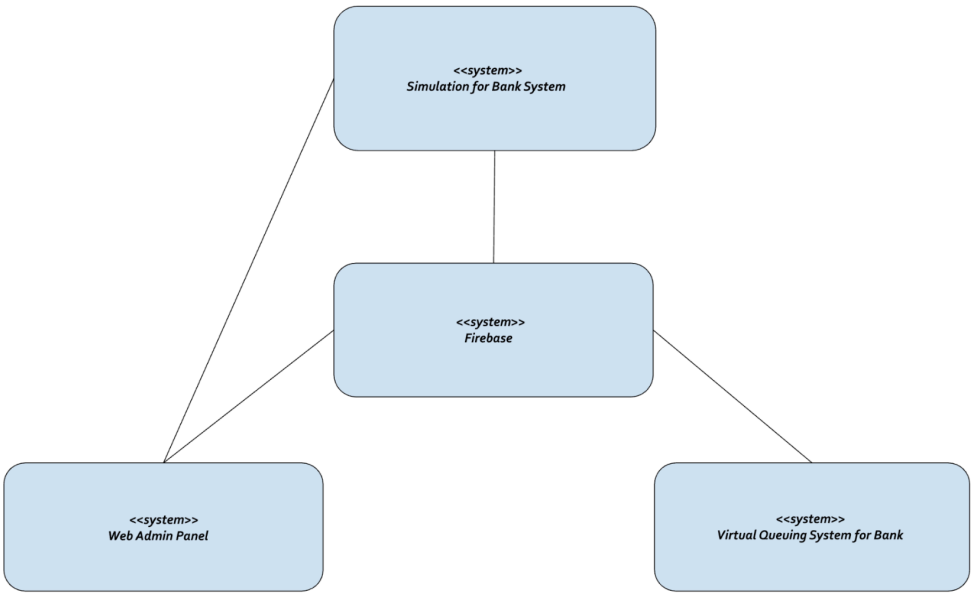



*Figure 1: Context diagram of virtual queuing system*

#### 1.5.1.1 System Interfaces

To function, this virtual queuing system for banks needs additional components and goods. These programs are Firebase, Simulation, and Web Admin Panel.

Firebase is a free platform developed by Google for developing mobile and web applications. It offers many different possibilities to its users. For this project, the possibilities offered by Firebase, shown below, will be utilised.

1. Firebase Store: Firebase Store is a cloud-based NoSQL database that keeps the data required in the application, and we keep users, branches, and operations and all details related to them.

2. Firebase Authentication: Firebase Authentication helps to authenticate the user and will be used in the login part of the project.

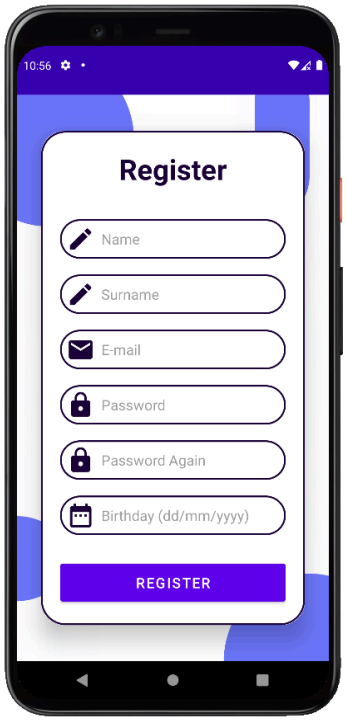
3. Simulation: Virtual Queue System gives some data from the Firebase into Simulation in order to analyze some criteria and metrics like waiting for time in queue or the system. The simulation system models Virtual Queuing System because of fetching real data from Firebase. Moreover, these simulation result metrics are shown by the admin web page, so regulations can be done by banks using this app according to these metrics. For example, if waiting time metrics are too much, it may be decided to increase the number of employees.

4. Web Admin Panel: The web admin panel connects with firebase and is used to view data such as customers, employees, bank branches, ticket information in the system over the web. It is also responsible for running the simulation from the web panel and showing the results such as waiting time to the administrator. In addition, operations such as changing customer priorities and adding employees can be done via the web admin panel.

### 1.5.1.2 User interfaces

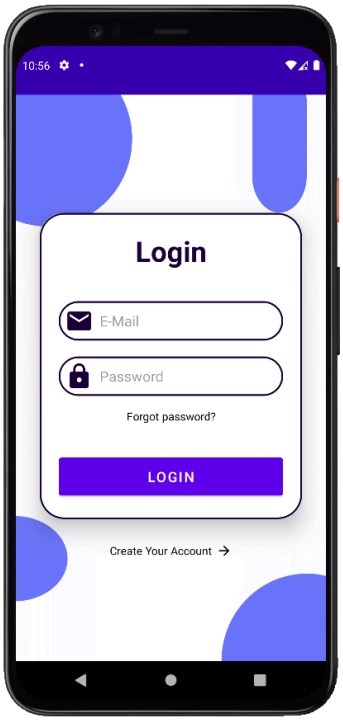
Our software product consists of a mobile application and web admin panel integrated with simulation. Here, we display the first mobile application screens. Then, we provide web admin panel screens. We explained what the screen stands for under the each screens.

Our mobile application  interfaces are the followings:



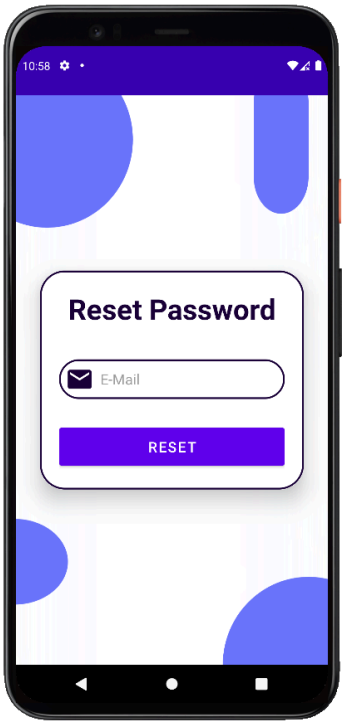

Register Screen

*Figure 2.0*



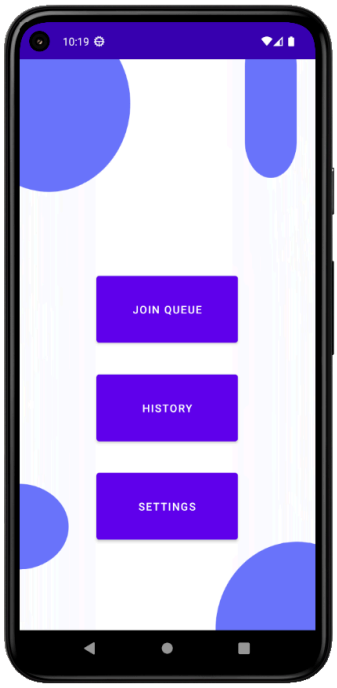

Login Screen

*Figure 2.1*



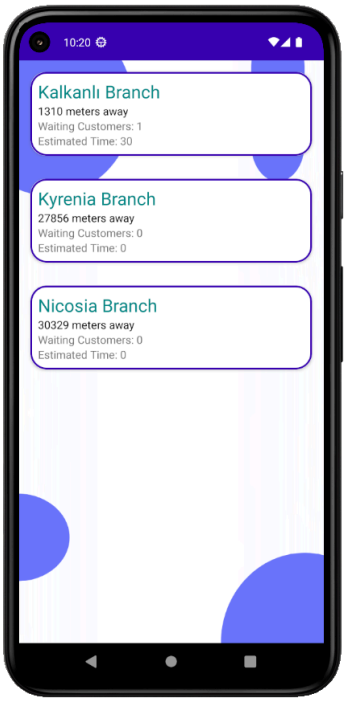

Forgot Password Screen

*Figure 2.2*



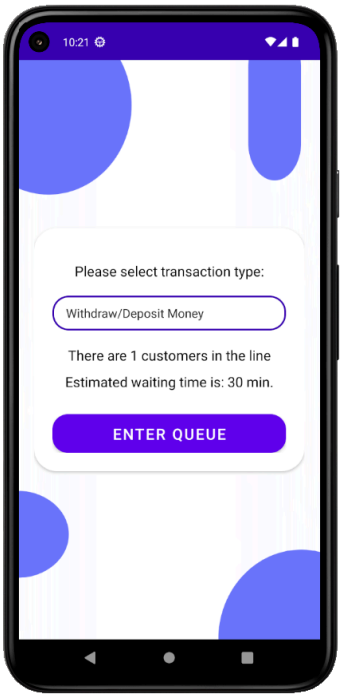

Customer - Menu

*Figure 2.3*



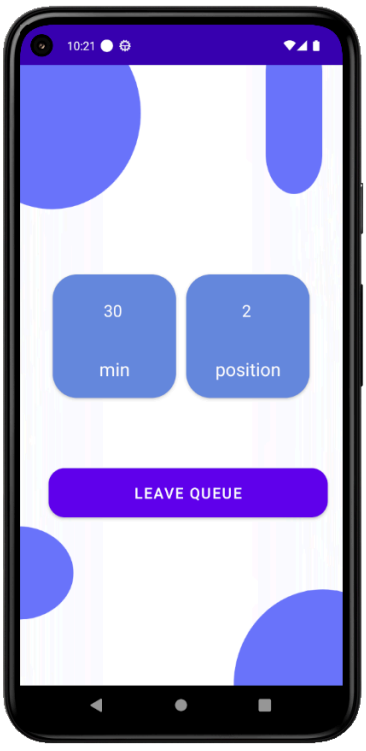

Customer - Select Branch

*Figure 2.4*



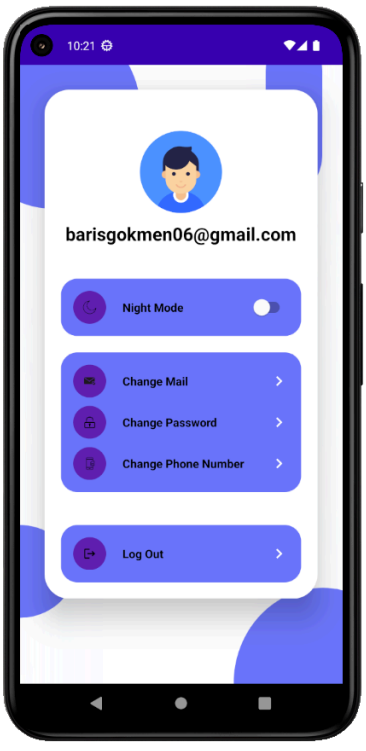

Customer - Entering Queue

*Figure 2.5*



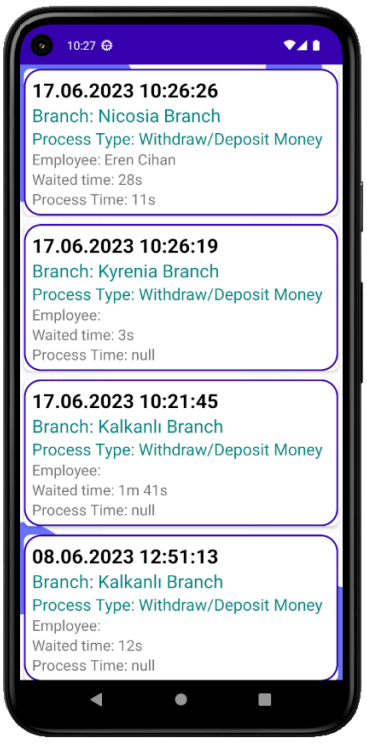

Customer - Active Queue

*Figure 2.6*




Settings

*Figure 2.7*




Customer - Ticket History

*Figure 2.8*

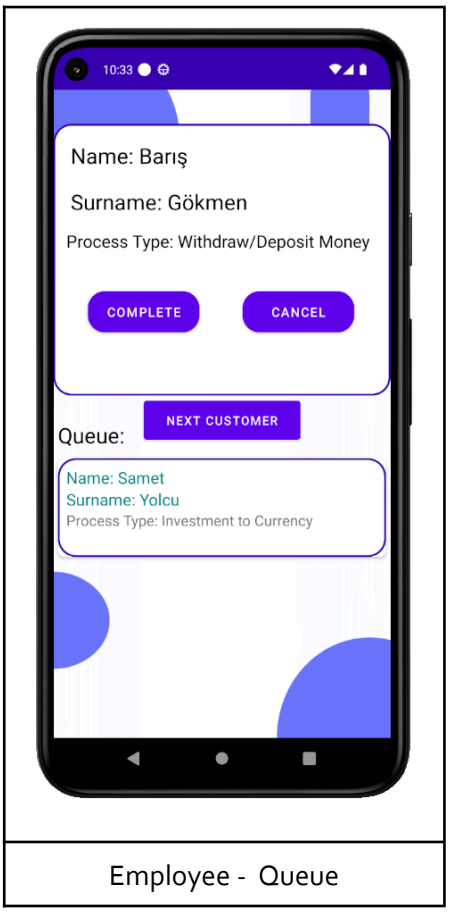



*Figure 2.9*

Our admin panel interfaces are the followings:

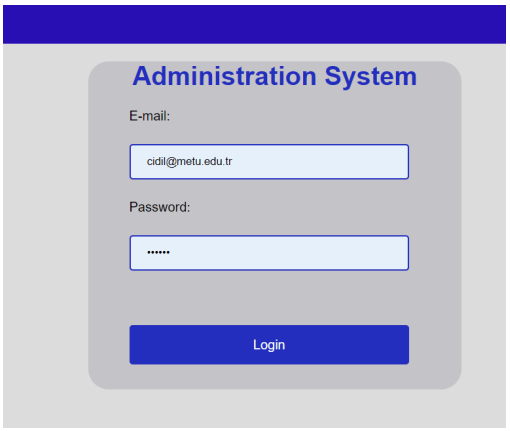



*Figure 2.10: Log in screen*

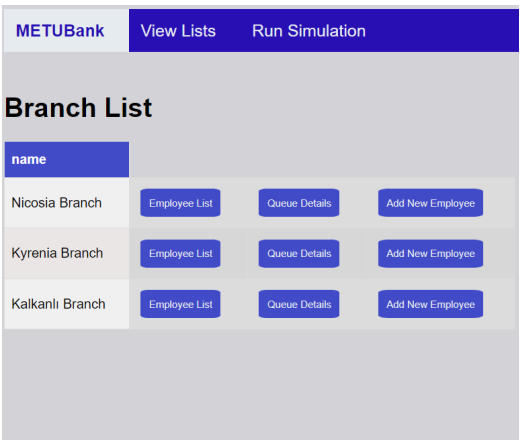



*Figure 2.11: Viewing Branch list*

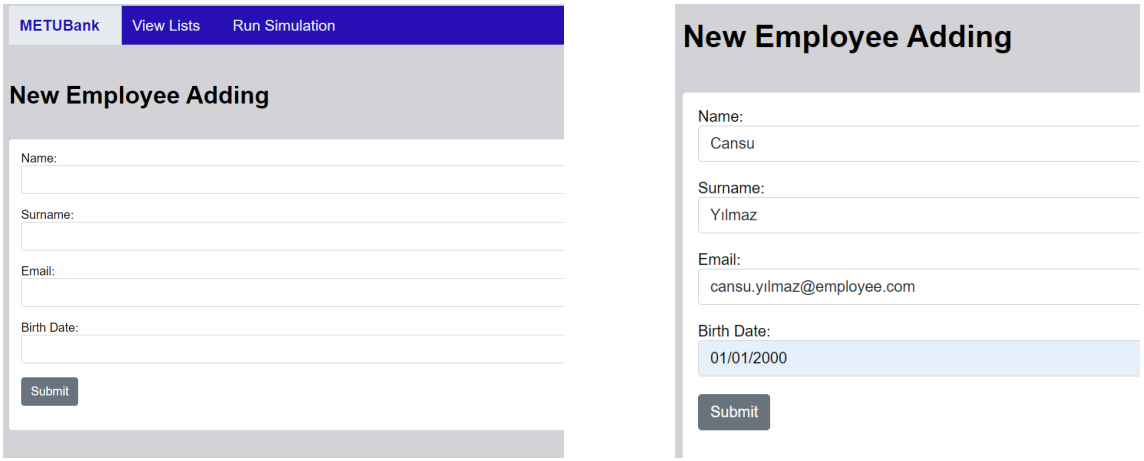



*Figure 2.12: Add new employee page*

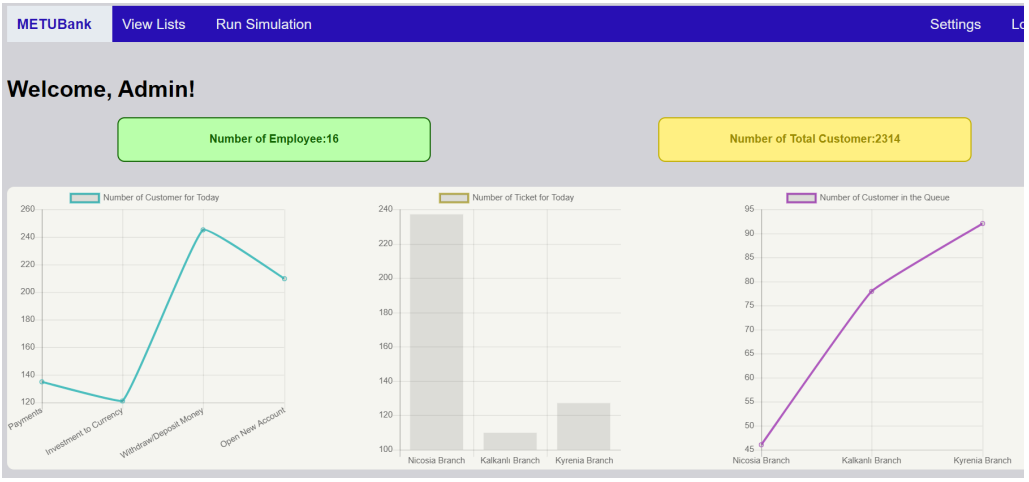



*Figure 2.13: Dashboard with statistics*


| name | surname | priority | email | age |
|---|---|---|---|---|
| Olivia | Jovi | 1 | olivia-jovi@gmail.com | 30 |
| Mehmet | Crawford | 2 | mehmet.crawford@gmail.com | 51 |
| Olivia | Simpson | 3 | olivia-simpson@gmail.com | 25 |
| Beyza | Kobain | 2 | beyza.kobain@gmail.com | 78 |
| Joe | Yilmaz | 3 | joe-yilmaz@gmail.com | 55 |
| Joe | Kobain | 2 | joekobain@gmail.com | 48 |
| Joan | Simpson | 3 | joan*simpson@gmail.com | 30 |
| Joe | Ferah | 2 | joe+ferah@gmail.com | 48 |
| Mehmet | Ferah | 1 | mehmet+ferah@gmail.com | 60 |
| John | Crawford | 2 | john-crawford@gmail.com | 85 |
| Beyza | Lennon | 3 | beyza.lennon@gmail.com | 50 |

*Figure 2.14: Viewing Customer list*

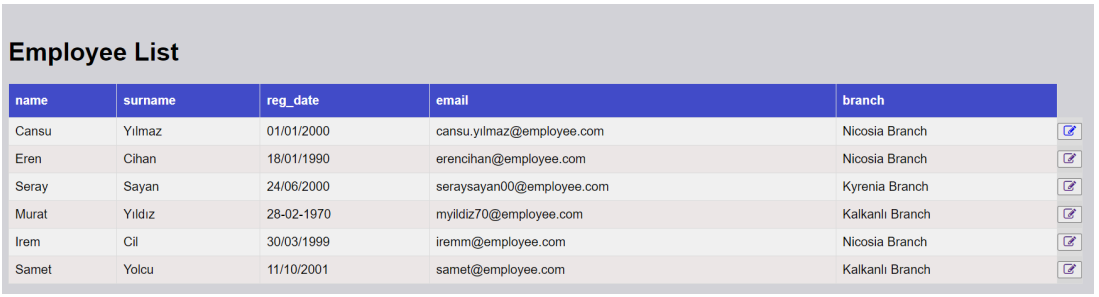



*Figure 2.15: Viewing Employee list*

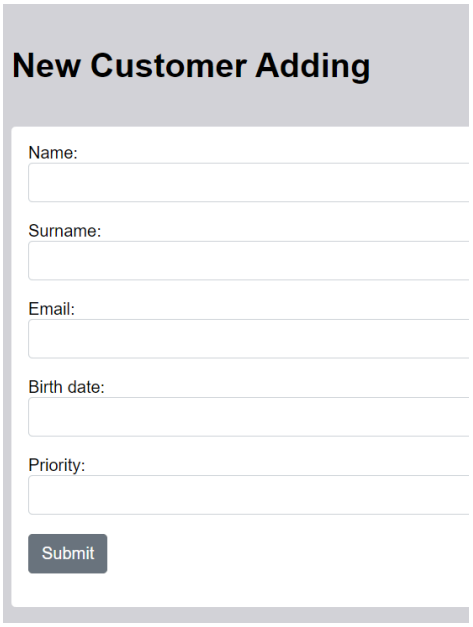



*Figure 2.16: Add new customer page*

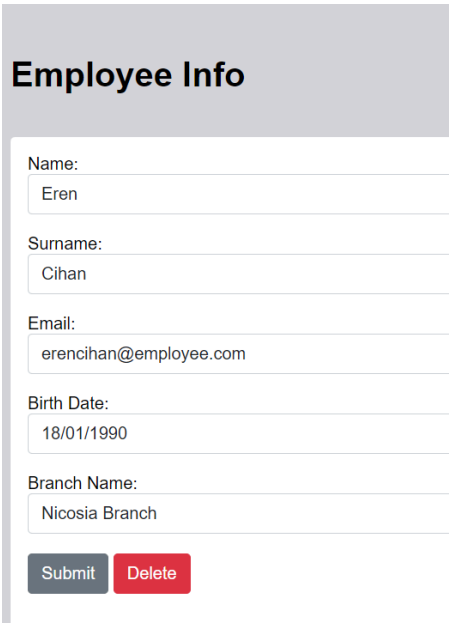



*Figure 2.17: Edit Employee page*

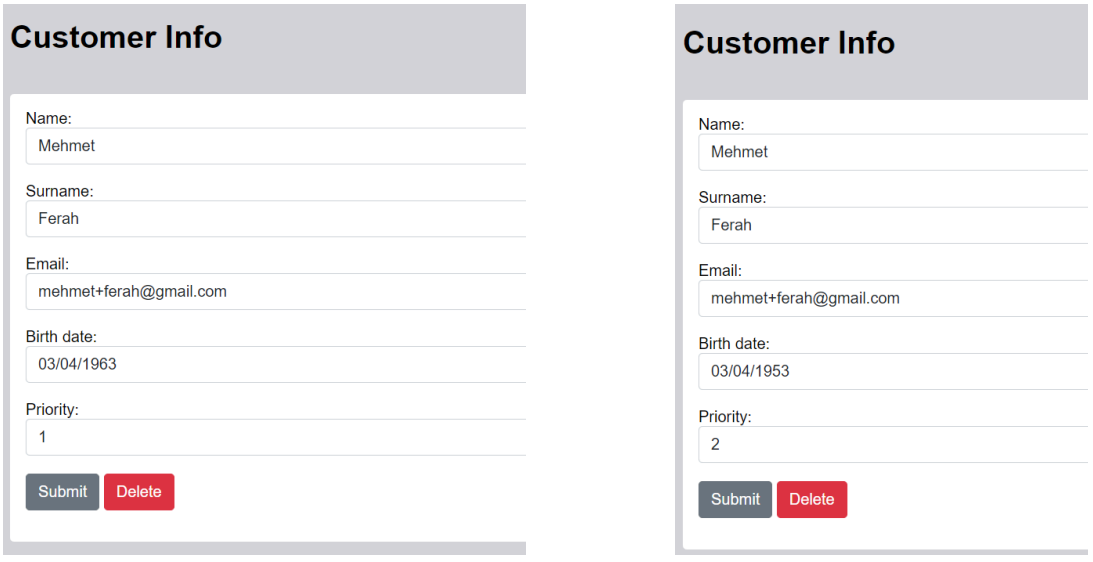



*Figure 2.18: Edit customer page*

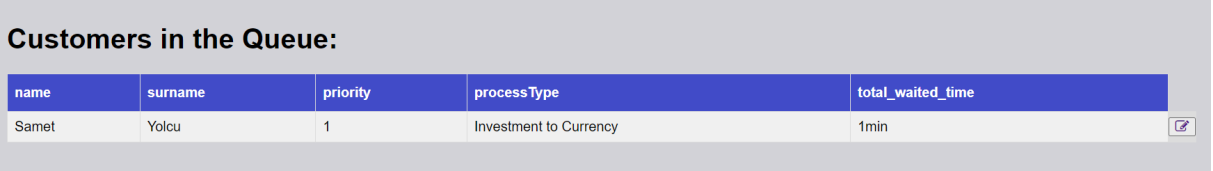



*Figure 2.19: Viewing Queue list*

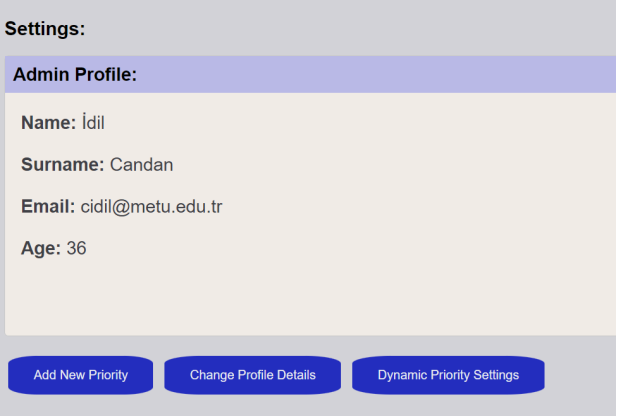



*Figure 2.20: Settings*

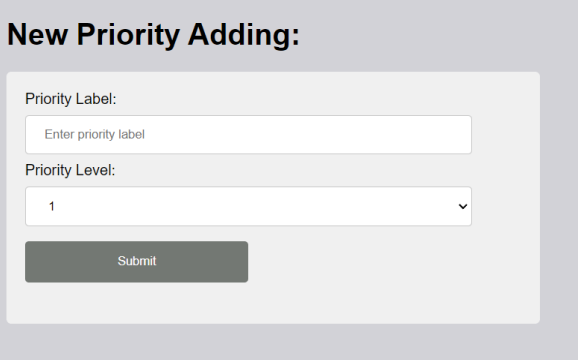



*Figure 2.21: New Priority*

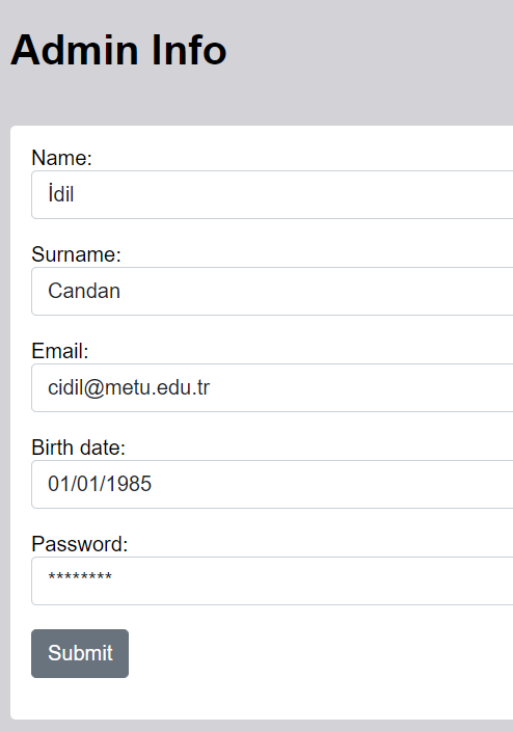



*Figure 2.22: Change Profile Details*


*Figure 2.23: Simulation Result Graph*

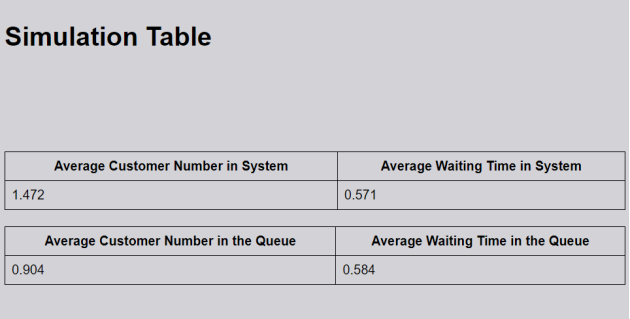



*Figure 2.24: Simulation Result Table*

1.5.2 Product functions

1. The system shall allow the creation of accounts for an unregistered user.
2. The system shall allow login after registration by entering email, password,name,surname and birthday information.
3. The system shall perform authentication of users of the application.
4. The system shall allow showing the branches list of a bank to users from near to far.
5. The system shall allow presenting the current queue of the branch to customers, employees, and project admin.
6. The system shall allow showing branch information to customers such as branch location .
7. The system shall allow customers to select any branch and take a queue from the branch.
8. The system shall allow customers to know about the current queue and send them alerts in the form of application notifications such as floating notifications from the upper bar.
9. The system shall allow employees to be informed about their customers and which one they want bank transactions such as payment with.
10. The system shall allow employees to change the state of the transactions, whether they are finished or cancelled.
11. The system shall allow access to bank statistics such as the total customer number per branch a day to the project admin.
12. The system shall allow adding new employees to specific branch by the project admin.
13. The system shall allow editing active queue details for each branch such as deleting a customer from the queue by the project admin.
14. The system shall allow changing the priority of the specific customer admin wants in the queue temporarily.
15. The system shall allow the admin to get information about some statistics like the waiting time in the queue and the waiting time in the system from the simulation and these statistics are calculated from the Firebase in order to analyze the Virtual Queuing System model.

1.5.3 Identified stakeholders and design concerns

We are aiming to develop a general purpose queueing system ,but since our model is a bank system, we provide the stakeholders according to our model.

Stakeholders

1. Bank Customers : Using the software to get better queue experience. There are registered customers who already have an account in our provided application, and there are also unregistered customers who just want to perform a specific bank

transaction without having an account in the application. For the unregistered ones, we are forcing them to register and add them in our application's database. Our application's database and bank database are different.

2. Bank Employees : Serving the Customers in the queue and system.

3. ProjectAdmin : Managing Branches, Employees and Customers and also running the Simulation.

### 1.5.4 User characteristics

Users which are customers, employees, and admin in our system. For the general characteristics, we used "user" below to avoid redundancy.

1. Admins should have discipline and observer skills.
2. Employees should have general knowledge about the queue algorithm.
3. Employees should have knowledge about managing the app functionalities.
4. Employees should be patient enough to serve a crowded group of customers.
5. Customers should have general ethics and respect the queue.
6. Users should have a mobile android device.
7. Users should have internet connection.

### 1.5.5 Limitations

1. This system is created only for a specific bank.
2. This system is used only by users who download the application to their phones.
3. This system only works when there is an internet connection.
4. Users can enter the queues only during the working hours of branches.
5. The planned mobile app accommodates Android devices as it is implemented by Android Studio.

### 1.5.6 Assumptions and dependencies

1. All users should have a mobile phone to be able to download and use the mobile application.
2. In case Firebase is deprecated, a new DB provider must be found for the software to continue functioning such as MongoDB, and IBM Cloudant.

# 2 Specific requirements

## 2.1 External interfaces

Here in Table 2, we display the needed inputs of Google Maps and Google Firebase as external interfaces and related outputs of them to the application.

| System Interface | Functionality | Input | Output |
|---|---|---|---|
| Web Admin Panel | Showing simulation results on a table and graph,<br>Viewing queue list details, customer and employee list,<br>Adding new priority feature, | Firebase collections (Branch,Employee,Ticket),<br>Simulation | Viewing some metrics like waiting in the queue and system on the table after running simulation,<br>Viewing some data (employee, branch,customer, and tickets data) from Firebase. |
| Simulation | Calculating some metrics like waiting time in the queue and system | Customer Number (Ticket number in Firebase),<br>Arrival Rate (Calculated by real arrival time data from Firebase),<br>Service Rate (Calculated by real service time data from Firebase),<br>Server Number (Number of Employee in application)<br>Priority List (based on Firebase Ticket) | Waiting time in the queue,<br>Waitin time in the system,<br>Average service time,<br>Total simulation time |
| Google Firebase | Database | User Information,<br>Queue information,<br>Counter information,<br>Branch Information,<br>User Login credentials | User Information,<br>Queue information,<br>Counter information,<br>Branch Information,<br>User Authentication Token |

*Table 2: External Interfaces*

## 2.2 Functions

In Figure 3, we utilised the Use Case Diagram to present the users, the needed functions of the application and their relations conveniently.

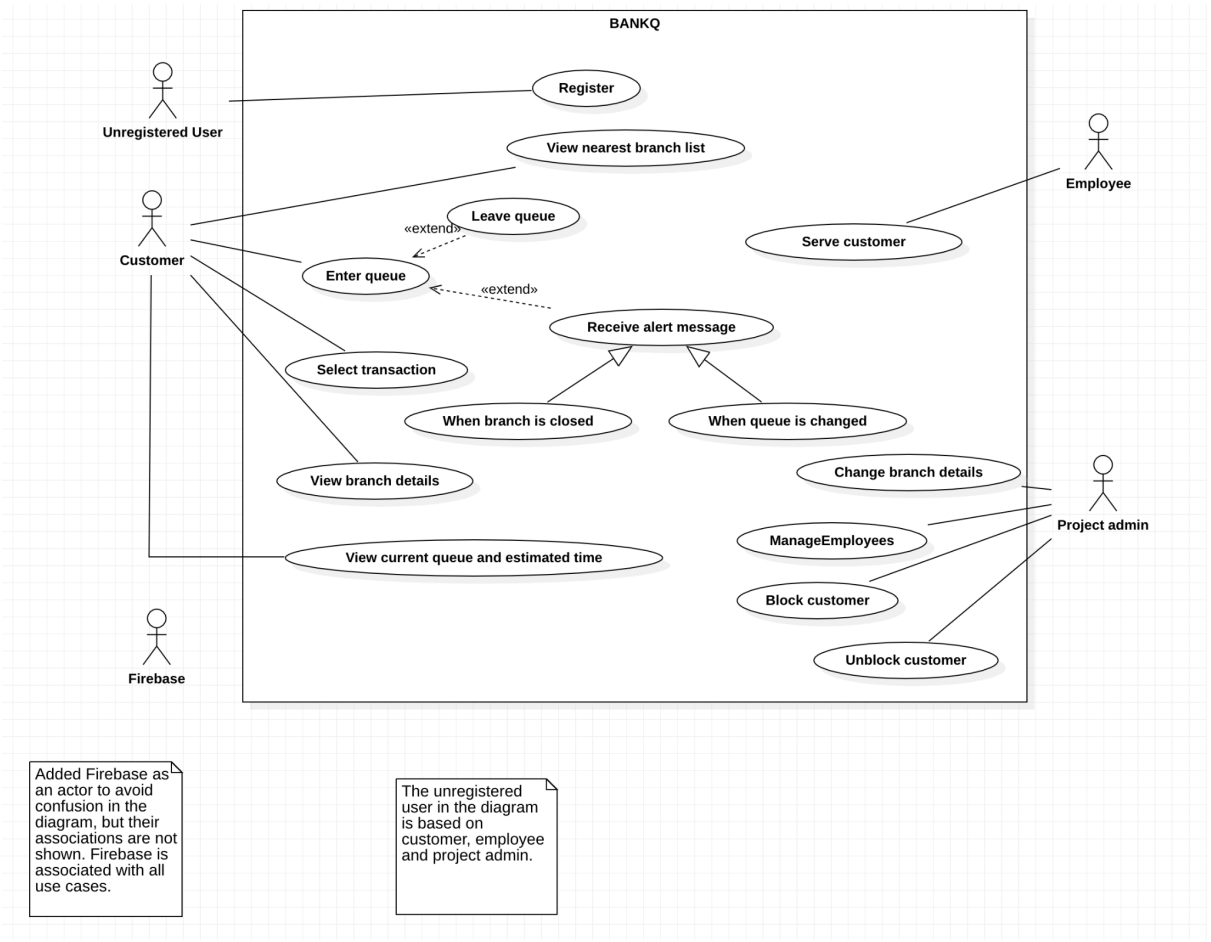



*Figure 3: Use case diagram*

1) This use case shows how the "Enter the queue" use case will be generated in the application.

| Use case | Enter the queue |
|---|---|
| **Actors** | Customer, Firebase |
| **Cross references** | Receive an alert message and select transactions. |
| **Typical Course of Events** | |
| **Actor Intentions** | **System Responsibility** |
| 1. Customer clicks enter the queue button. | |
| | 2. System shows transactions (such as withdrawal, deposit, checks, payment, debit card charges etc.) for selection. |
| 3. Customer selects one of them. | |
| | 4.The system sends customer information to Firebase. |
| 5. Firebase takes customer details and saves them. | |
| 6. Firebase sends a confirmation message to the system. | |
| | 7. System sends confirmation and queue details to customer. |
| Alternative Courses | |
| Step 6: If the customer wants to enter the queue for the same transaction, system reask customer selection, so returns to Step3. | |
| Step 7: If entering the queue case is done successfully, receive the alert message will be applied. (see Use Case "Receive Alert Message"). | |

2) This use case shows how the "Leave the queue" use case will be generated in the application.

| Use case | Leave the queue |
|---|---|
| **Actors** | Customer, Firebase |
| **Cross references** | |
| **Typical Course of Events** | |
| **Actor Intentions** | **System Responsibility** |
| 1. Customer clicks leave the queue button. | |
| | 2. System sends customer information to Firebase. |
| 3. Firebase takes customer details and changes the state of it as unsuccessful. | |
| | 4. The system sends customer information to Firebase. |
| 5. Firebase sends notification to system. | |
| | 6. System sends confirmation message to customer. |
| **Alternative Courses** | |

3) This use case shows how the "Select transactions" use case will be generated in the application.Transaction types such as deposit, withdrawal, loan payment are predefined.

| Use case | Select transactions |
|---|---|
| Actors | Customer, Firebase |
| Cross references | |
| Typical Course of Events | |
| Actor Intentions | System Responsibility |
| | 1. System shows the type of transactions. |
| 2. Customer selects one of them. | |
| | 3. System computes valid queue no for customer. |
| | 4. System sends these informations to Firebase. |
| 5. Firebase takes details and saves them. | |
| 6. Firebase sends notification to the system. | |
| | 6. System sends a notification with queue no to the customer. |
| Alternative Courses | |

4) This use case shows how the "View nearest branch list" use case will be generated in the application.

| Use case | View nearest branch list |
|---|---|
| Actors | Customer, Firebase |
| Cross references | |
| Typical Course of Events | |
| Actor Intentions | System Responsibility |
| 1.Customer clicks "Enter queue". | |
| | 2.System asks customer to open GPS attribute from their mobile phone. |
| 3. Customer opens GPS attribute in his/her mobile device. | |
| | 3. System takes Branch locations from Firebase db. |
| | 4.System calculates the distance between mobile device and branch locations. |
| | 5. System shows the branch list with the measurement details. |

5) This use case shows how the "View current queue and estimated time" use case will be generated in the application.

| Use case | View current queue and estimated time |
|---|---|
| **Actors** | Customer, Firebase |
| **Cross references** | |
| **Typical Course of Events** | |
| **Actor Intentions** | **System Responsibility** |
| 1.Customer clicks queue details button | |
| | 2. System calculates the current remaining time and send to the Firebase |
| 3. Firebase takes and updates it. | |
| 4.Firebase sends the estimated time and current queue details to the system. | |
| | 5.System shows the details to the user. |

6) This use case shows how the "Serve Customer" use case will be generated in the application.

| Use case | Serve Customer |
|---|---|
| **Actors** | Emloyee, Firebase |
| **Cross references** | |
| **Typical Course of Events** | |
| **Actor Intentions** | **System Responsibility** |
| 1. Employee clicks view queue details button. | |
| | 2. System shows the types of operations. |
| 3. Employee clicks update customer details button. | |
| | 4. System shows the information that can be updated. |
| 5. Employee clicks status button and sets it successful. | |
| | 6. System takes this detail and send it to Firebase. |
| 7. Firebase takes and saves this details. | |
| **Alternative Courses** | |
| Step 5: If the customer does not come to the bank counter, employee sets it unsuccessful. | |

2.3 Usability Requirements

1. The system shall be designed clearly so that the system will be as practical as possible and

the users of the application do not face any obstacles.

2. The system shall be fast enough to update queues on time.

3. The system shall have a good security system in order to protect personal data.

4. The application shall be simple enough to utilise for every user between 18-85 years old.

2.4 Performance requirements

1. The system shall handle the total number of concurrent users estimated from the bank's

system history with no drop in performance.

2. Since we are developing a queuing system to make these bank queues faster, our provided

system needs to be fast to manage transactions.

3. Since the information of the users will be encrypted before saving to the Firebase, the

system shall encrypt and decrypt the data quickly.

## 2.5 Logical database requirements

In fact, we are planning to use Firebase which is a cloud based NoSQL database. However,here we wanted to show the data planned to store in our database with an Entity

Relationship Diagram to visualise our data flow efficiently.

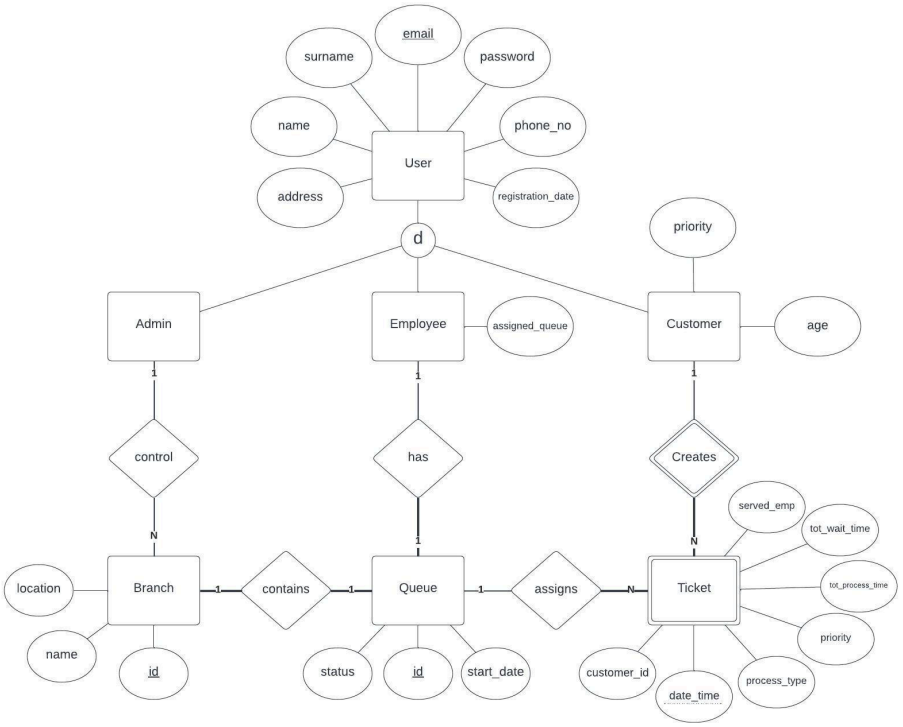



*Figure 4:Entity Relationship Diagram*

1. Customer creates a Ticket when entering a queue. Customer queue details stored inTickets.

2. "email" is unique for all accounts. If a person is anAdmin and a Customer at the same time, s/he will have different accounts.

3. "process_type" attribute ofTicket is indicating the transaction type of a queue.

## 2.6 Software system attributes

- Reliability

1. The system shall secure the database and handle server-side errors in case of any

system failure. Since we use a realtime-database and in case of a system failure such

as class exceptions, system crashes the database, it will affect whole clients.

2. The system shall run both on WI-FI and cellular data since the usage of the application is location independent. This means that the users may use the application in the areas that have noWI-FI. In this case, they could use cellular data.

- Availability

  1. The system shall record and provide statistics instantly as the system works real-time.

  2. Queue functionalities shall be available during work hours since in working hours, there can be a queue at any time.

  3. The system shall be available for 7/24 for branch and user information. Since even though the queue functionalities will be offline, users may want to check their past records.

- Security

  1. Only the admin shall have access to employees' information. Since if another employee or customers would have access to an employee's information, it is a violation of personal data protection law.

  2. The system shall encrypt user passwords and user information in order to keep the critical information safe in case of any system leak.

  3. The system shall be secured against DDos attacks, since the system works in real-time and a DDos attack may increase the latency significantly.

- Maintainability

  1. The system shall be maintainable during working because it works in real-time and any maintenance break would prevent the whole system from being used.

  2. The system shall protect the database and its content during maintenance since the database is critical for the system.

- Portability

  1. The system shall be available on android devices, since the client side app will be an android app.

  2. The system shall consume power as low as possible. Because the android app will be active during the queue and it is not certain how long it will take.

2.7 Supporting information

1. Documentation about Firebase and its useability, and the products of Firebase.

(n.d.). Google Firebase Documentation. Google Firebase. Retrieved January 11, 2023, from https://firebase.google.com/docs

2. Explanation about the COCOMO cost estimation model, the way it is used, and examples of its implementation.

Software Engineering | COCOMO Model - javatpoint. (n.d.-b).www.javatpoint.com.

Retrieved from: https://www.javatpoint.com/cocomo-model

3. Supporting information about how to useTeamGantt and how to start creating a gantt chart.

TeamGantt Support (n.d.).Getting Started toTeamGantt.TeamGantt. Retrieved January11,2023,from

https://support.teamgantt.com/article/77-welcome-to-teamgantt

4. Definition and explanation of Click-Wrap and Shrink-Wrap agreements, and the areas they are used.

The Origin of Click-Wrap: Software Shrink-WrapAgreements. (2000, March 22).

WilmerHale. Retrieved from:

https://www.wilmerhale.com/insights/publications/the-origin-of-click-wrap-software-shrink-

wrap-agreements-march-22-2000

5. Documentation about Functional Point (FP)Analysis and examples about its implementation.

Software Engineering | Functional Point (FP)Analysis - javatpoint. (n.d.).

www.javatpoint.com. Retrieved from:

https://www.javatpoint.com/software-engineering-functional-point-fp-analysis

6. An article about the advantages of cloud functions and how to use them.

Temel, K.A. (2017, September 18). Firebase Cloud Functions — Cihazdan Cihaza Bildirim Gönderimi. Medium. Retrieved January 11, 2023, from

https://kurtulusahmet.medium.com/firebase-cloud-functions-cihazdan-cihaza-bildirim-g%C3%B6nderimi-fd7e992ea9fd#:~:text=Firebase%20Cloud%20Functions%2C%20Google%20Cloud,projeniz%20i%C3%A7in%20%C3%A7al%C4%B1%C5%9Fan%20bir%20yap%C4%B1d%C4%B1r.

7. An article about the advantages of FirebaseAuth which is part of Firebase and how to use them.

Yılmaz, S. (2016, October 18).Android FirebaseAuthentication (Email&Parola veGoogle).Medium. Retrieved January 11, 2023, from

https://sinanyilmaz.medium.com/android-firebase-authentication-email-parola-ve-google-cc154d1bf5fe#:~:text=Firebase%20Authentication%2C%20sa%C4%9Flad%C

4%B1%C4%9F%C4%B1%20SDK'lar%C4%B1yla,Anonim%20olarak%20kimlik%20do%C4%9Frulamay%C4%B1%20destekler.

8. An article about usage of Cloud Firestore and its comparison withGoogle RealTime

Firebase.Göktaş,A. (2019,June 23).Android Cloud Firestore RESTAPI. Medium. Retrieved January 11, 2023, from

https://aligoktass.medium.com/android-cloud-firestore-rest-api-ile-firebase-k%C3%BCt%C3%BCphanesi-eklemeden-database-kullan%C4%B1m%C4%B1-d461478a1f6c

# 3 Software Estimation

## Function Point Estimation

In Figure 5.0, and Figure 5.1, program characteristics are listed.

In Figure 5.2, the coefficients according to program characteristic types and complexity.

| |
|---|
| Inputs; |
| Register to the app — high Complexity<br>List branches on Google Maps according to customer's location — high Complexity<br>Select ENTER while entering Queue — Low Complexity<br>Login to the system — Low Complexity<br>Update branch details when needed – Medium Complexity<br>Select view statistics – Low Complexity<br>Mark customer status -Low complexity<br>Close bank counter- Low complexity<br>Open bank counter- Low Complexity<br>Block/unblock customer-Low complexity |
| In Total : 7 x Low Complexity , 1 x Medium Complexity ,2 x High Complexity |
| |
| Outputs; |
| Display a Map with the customers' and branches' locations — High Complexity<br>List of Branches with queue status, distance with the customer, tel number etc. — Medium Complexity<br>Alert message when needed - Low complexity<br>Current Queue and estimated time will be displayed dynamically — High Complexity |
| In Total : 1x Low Complexity,1 x Medium Complexity , 2 x High |

*Figure 5.0*

| |
|---|
| Inquiries; |
| Registration process — High Complexity |
| List the Branches — Medium Complexity |
| Calculate the distance between customer and branches— Low Complexity |
| Enter to a queue — High Complexity |
| Leave the queue — Medium Complexity |
| Update the users info in cloud— High Complexity |
| Update the queues info in cloud— High Complexity |
| Update branch details in cloud — Medium Complexity |
| Show statistics function for branch owners— High Complexity |
| In Total : 1 x Low Complexity , 3x Medium Complexity , 5 x High |
| |
| Logical Data Files; None since we keep all the data in Firebase |
| |
| External Interface Files; |
| Customers Collection — Medium Complexity |
| Employees Collection — Medium Complexity |
| Branches Collection — Medium Complexity |
| Queues Collection — High Complexity |
| In Total : 3 x Medium Complexity , 1 x High Complexity |

*Figure 5.1*

| | Function Points | | |
|---|---|---|---|
| Program Characteristic | Low Complexity | Medium Complexity | High Complexity |
| Number of inputs | x 3 | x 4 | x 6 |
| Number of outputs | x 4 | x 5 | x 7 |
| Inquiries | x 3 | x 4 | x 6 |
| Logical internal files | x 7 | x 10 | x 15 |
| External interface files | x 5 | x 7 | x 10 |

Figure 5.2

**UTFP:**

Inputs : 7 * 3(Low) + 1 * 4(Medium) + 2 * 6(High)= 37

Outputs : 1* 4(Low)+ 1 * 5(Medium) + 2 * 7(High) = 23

Inquiries : 1* 3(Low) + 3 * 4(Medium) + 5 * 6(High) = 45

Logical Internal Files : Not Exist = 0

External Interface Files : 3 * 7(Medium) + 1 * 10(High) = 31

UTFP = 43 + 23 + 35 + 0 + 31 = **136**

**ATFP:**

Total of Degree of Influence = 46

VAF = (TDI)*0.01+0.65 = 46*0.01+0.65 = **1.11**

ATFP = UTFP * VAF = 136*1.11 = **150.96**

**LOC:**

We are using Kotlin for our project and since it is not on the list, the calculations will

be based on OOP Default.

Language Unit Size= 29 for Kotlin

LOC = ATFP * Language Unit Size = 150.96*29 = **4377.84**

In figure 5.3, General system characteristics are listed with Degree of Influences. Also, VAF, TDI, UTFP, and ATFP are displayed.

| General System Characteristics (GPC) | Degree of Influence (DI) – 0:least, 5: most | Explanation |
|---|---|---|
| Data Communications | 3 | How many communication facilities are there to aid in the transfer or exchange of information with the application or system? |
| Distributed Processing | 3 | How are distributed data and processing functions handled? |
| Performance | 4 | Did the user require response time or throughput? |
| Heavily used configuration | 3 | How heavily used is the current hardware platform where the application will be executed? |
| Transaction Rates | 5 | How frequently are transactions executed daily, weekly, monthly, etc.? |
| On line data entry | 4 | What percentage of the information is entered On-Line? |
| Design for end-user efficiency | 5 | Was the application designed for end-user efficiency? |
| Online updates | 4 | How many internal logical files are updated by online transaction? |
| Complex processing | 4 | Does the application have extensive logical or mathematical processing? |
| Usable in other applications | 3 | Was the application developed to meet one or many user's needs? |
| Installation ease | 2 | How difficult is conversion and installation? |
| Operational ease | 3 | How effective and/or automated are start-up, back up, and recovery procedures? |
| Multiple sites | 0 | Was the application specifically designed, developed, and supported to be installed at multiple sites for multiple organizations? |
| Facilitate change | 3 | Was the application specifically designed, developed, and supported to facilitate change? |
| **Total of Degree of Influence** | **46** | |
| **Value adjustment factor (VAF)** | **1.11** | |

| Program Characteristics | Low Complexity | Medium Complexity | High Complexity | Total |
|---|---|---|---|---|
| Inputs | 7 | 1 | 2 | 37 |
| Outputs | 1 | 1 | 2 | 23 |
| Inquiries | 1 | 3 | 5 | 45 |
| Logical Internal Files | 0 | 0 | 0 | 0 |
| External Interface Files | 0 | 3 | 1 | 31 |
| | | | **Unadjusted Total of Function Points (UTFP)** | **136** |
| | | | **Adjusted Total of Function Points (ATFP)** | **150.96** |

*Figure 5.3 : General system characteristics*

**Constructive Cost Model COCOMO**

Our project is not a big project, it is a small project which can be done by 4 people. Also, we have a few innovations compared to similar projects, and we will use Android Studio as a development environment which is very stable. That's why we chose Organic development mode.

In Figure 5.4, project characteristics of Organic development mode are shown and related coefficients are listed.

| Development Mode | Project Characteristics | | |
|---|---|---|---|
| | Size | Innovation | Deadline/constraints | Dev. Environment |
| Organic | Small | Little | Not tight | Stable |

| Basic COCOMO | a | b | c |
|---|---|---|---|
| Organic | 2.4 | 1.05 | 0.38 |
| Semi-detached | 3.0 | 1.12 | 0.35 |
| Embedded | 3.6 | 1.20 | 0.32 |

**Model Coefficients**

*Figure 5.4 : Project characteristics*

KDSI = ATFP * Language Unit Size/ 1000 = 4377.84/1000 ~= **4.38**

For Organic development Mode:

Man-Months = 2.4 * (KDSI)^(1.05) = 2.4 * (4.38)^(1.05) = **11.32**

TDEV = 2.5 * (MM)^(0.38) = **6.29**

For estimated team size:

MM / TDEV = 11.32 / 6.29 = 1.7996 ~= **2 people**

| | LANGUAGE | LEVEL | AVERAGE SOURCE STATEMENTS PER FUNCTION |
|---|---|---|---|
| **For Kotlin:** | Object-Oriented default | 11 | 29 |

In Figure 5.5, Language Unit Size's, LOC's and KDSI's results are shown according to Kotlin. Also, MM and TDEV are displayed for Organic type.

| Programming Language | Percentage in Project | Language Unit Size | Language Unit Size in Project |
|---|---|---|---|
| Kotlin | 1 | 29 | 29 |
| | | | 0 |
| | | | 0 |
| | | | 0 |
| | | Language Unit Size | 29 |
| | | Lines of Code | 4377.84 |
| | | Kilo Delivered Source Instruction (KDSI) | 4.38 |

| Development Type | Organic |
|---|---|
| Estimated Effort in Man-Months | 11.32 |
| Estimated Development Time in Month | 6.29 |
| Estimated Team Size | 2 |

*Figure 5.5*

**Jones's First-Order Effort Estimation**

In Figure 5.6, coefficients are displayed according to the kind of software and Software Organization's skills.

```
Estimated Effort = (ATFP^(3*Class Exponent))/27
```

| | ← Software Organization's Skills/Abilities → | | |
|---|---|---|---|
| Kind of Software | Best in Class | Average | Worst in Class |
| Systems | 0.43 | 0.45 | 0.48 |
| Business | 0.41 | 0.43 | 0.46 |
| Shrink-wrap | 0.39 | 0.42 | 0.45 |

*Figure 5.6*

Shrinkwrap Software means Software licensed to a Company under a shrink-wrap or click-through agreement on reasonable terms through commercial distributors or in consumer retail stores. We are planning to make an android app which is similar to the shrink-wrap software.

Estimated Effort for Shrink-Wrap:

Best in Class = [ (4377.84) ^ (3*0.39)]/27] =**13.12**

Average= [(4377.84) ^ (3*0.42)/27] = **20.61**

Worst in Class=[( 4377.84) ^ (3*0.45)/27] = **32.37**

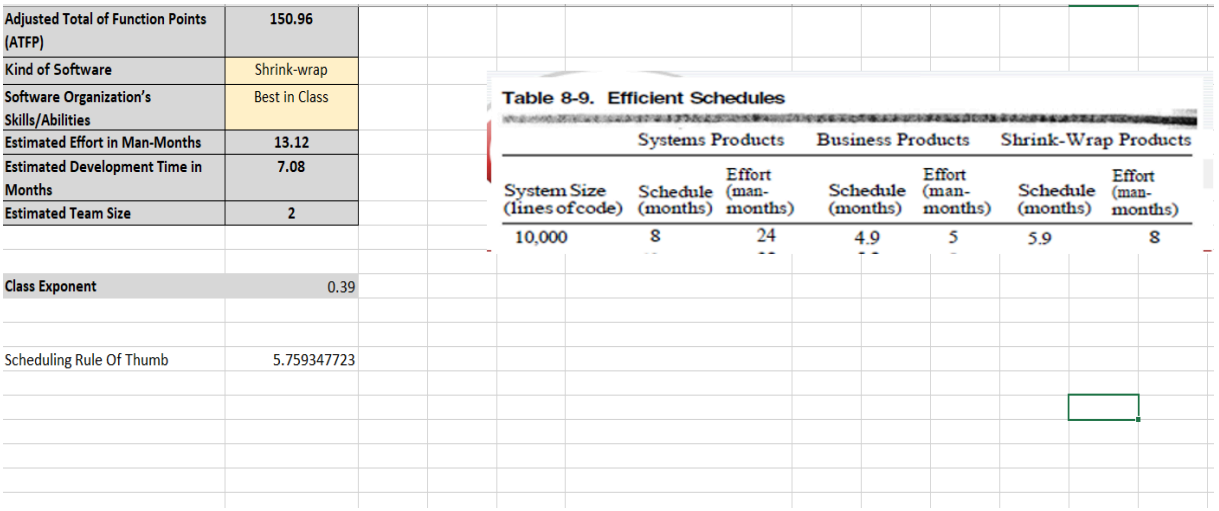
Rough Schedule Estimate = (ATFP)^Class Exponent :

Best in Class = (4377.84) ^ (0.39) = **7.08**

Average= (4377.84) ^ (0.42) = **8.22**

Worst in Class= (4377.84) ^ (0.45) = **9.56**

In Figure 5.7, Jones's First-Order Effort Estimation and Estimation Tables' results are presented. We assume that we are using better software skills and organisations, so we selected best in class. Also, we assume that we are doing most of the things right, but the efficient schedules stop short of making the ideal-case assumptions that underlie the shortest possible schedules, and efficient schedule is the more accurate one for our project. That's why we chose Efficient Schedules.



| | |
|---|---|
| Adjusted Total of Function Points (ATFP) | 150.96 |
| Kind of Software | Shrink-wrap |
| Software Organization's Skills/Abilities | Best in Class |
| Estimated Effort in Man-Months | 13.12 |
| Estimated Development Time in Months | 7.08 |
| Estimated Team Size | 2 |
| Class Exponent | 0.39 |
| Scheduling Rule Of Thumb | 5.759347723 |

**Table 8-9. Efficient Schedules**

| | Systems Products | | Business Products | | Shrink-Wrap Products | |
|---|---|---|---|---|---|---|
| System Size (lines of code) | Schedule (months) | Effort (man-months) | Schedule (months) | Effort (man-months) | Schedule (months) | Effort (man-months) |
| 10,000 | 8 | 24 | 4.9 | 5 | 5.9 | 8 |

*Figure 5.7 :* Jones's first-order effort estimation table

We employed COCOMO Cost Estimation, Jones's First-Order Effort Estimation and Estimation Tables to correctly estimate the duration of our product's development, and how many people it will take in order to implement it.

According to COCOMO results, it should take 6.29 development time in Man-Months and approximately 2 people to achieve our application. For Jones's First-Order Effort Estimation we need 7.08 development time in Man-Months and 2 people to implement our product. Lasty, as Estimation Tables present when we choose Shrink- Wrap software organisation and 10,000 lines of code, it will take 5.9 Schedule in months and 8 Effort in Man-Months to complete our product. As you can see from these results, we need roughly 7 months, and 2 people to accomplish our project. Since we are undergraduate students and we have other courses and responsibilities, it makes sense to complete our project with 4 people instead of 2 people, and finish it in 1 school year which takes 9 months instead 7 months.

As a conclusion, our product can be done in 9 months with a team of 4 people as we are planning to do.

# 4 Architectural Views

## 4.1 Logical View

*The logical view shows the key abstractions in the system as object classes.*

### 4.1.1 Class Diagram

Figure 6 shows our class diagram for Queue Management System Project. We mainly used aggregation and composition since we want to keep track of objects as lists in each other. Since our project must record all of the entries and lists.

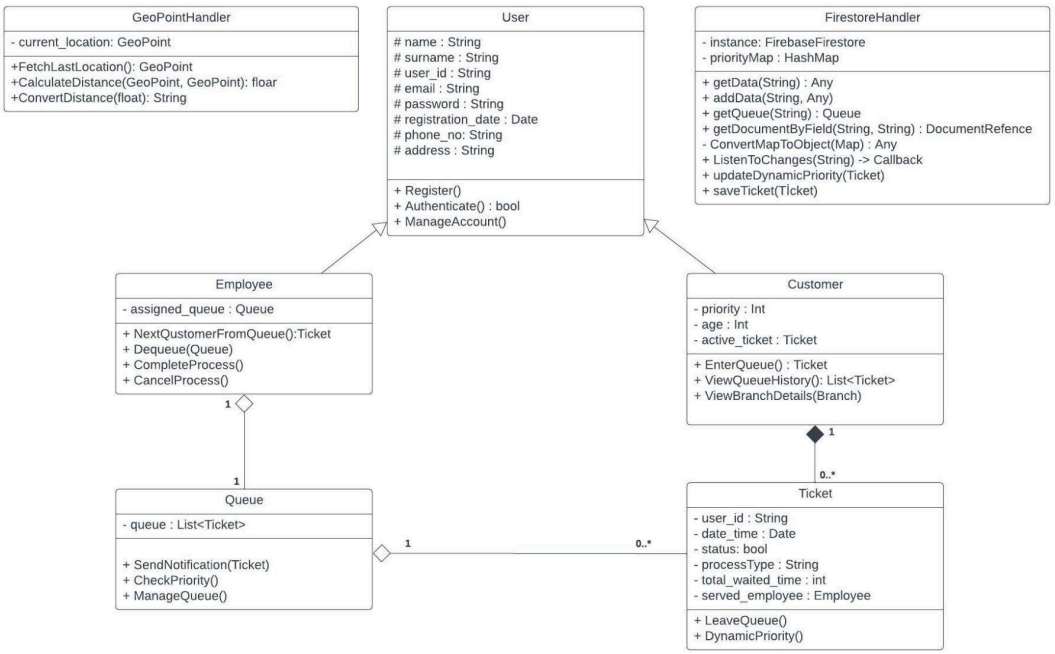



*Figure 6 : UML Class Diagram of the Project*

1. password in String kept encrypted by our database, Firestore.

2. All ID's are in string form inGoogle Firebase.

3. getData and addData methods can work with any class.

4. getDocumentByField method finds and returns DocumentReference type by appropriate field and value.

5. ConvertMapToObject method converts the data from firebase no-sql format to our objects.

6. ListenToChanges is a custom listener that calls back the changes.

7. GeoPoint handler is an interface API by Android OS. It fetches the location as GeoPoint.

## 4.2 Process View

The process view shows how the system is composed of interacting processes.

### 4.2.1 Activity Diagram

Since our system has multiple use-cases for four different actors, we only provide the critical three activity diagram below.

1. Select transaction

In Figure 6, the diagram consists of events occurring one after another. There is no decision. Therefore, we did not use any decision, fork or join in this diagram. We only provide the flow of actions.

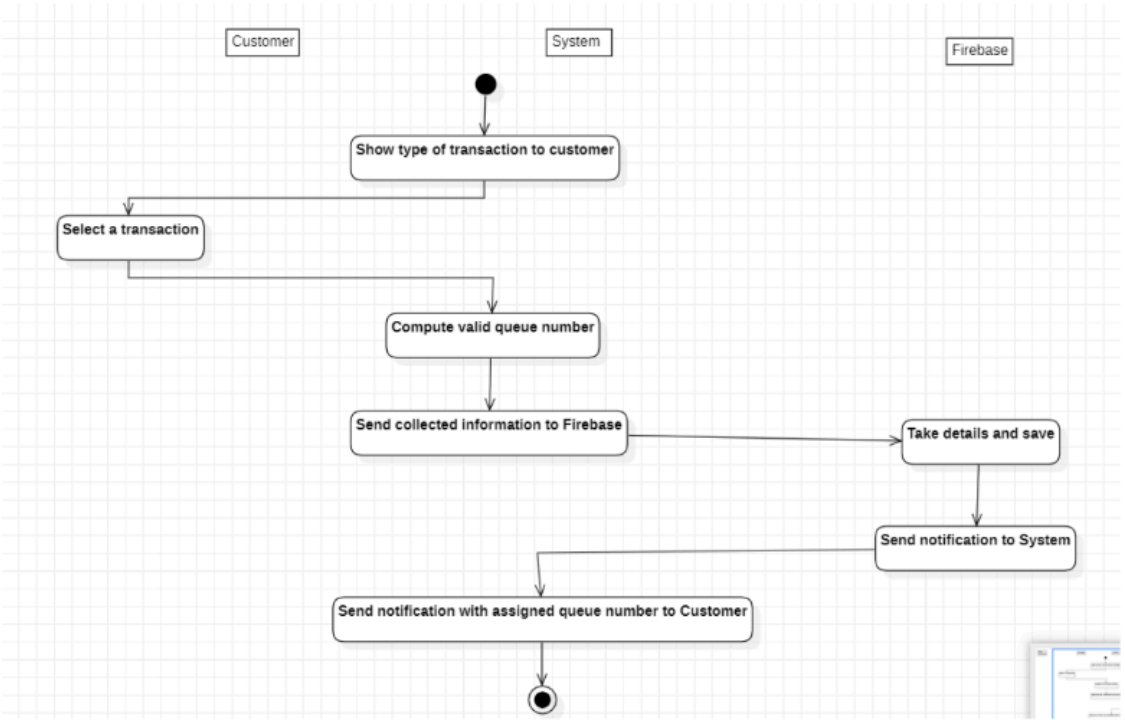



*Figure 7 :Activity diagram of select transaction*

2. Enter queue

In Figure 8, we only provide one fork and join at the end as sending confirmation message and queue details to Customer needs to be done at the same time. These two actions do not wait for each other to start.

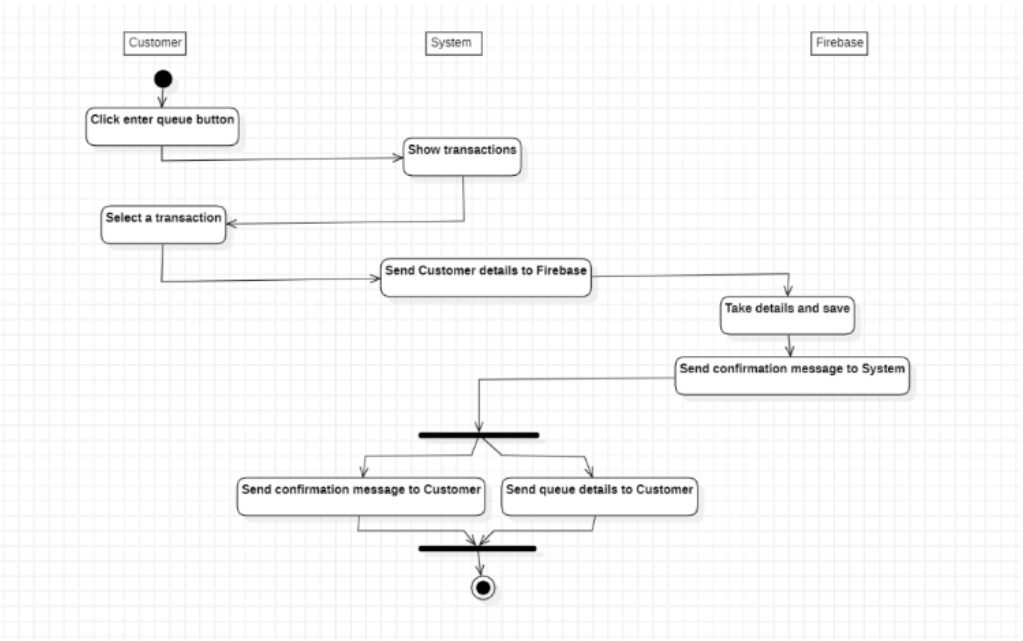



*Figure 8: Activity diagram of enter queue*

3. View current queue and estimated time

In Figure 9, we only provide one fork and join as sending estimated time and queue details to Customer needs to be done at the same time. These two actions do not wait for each other to start.

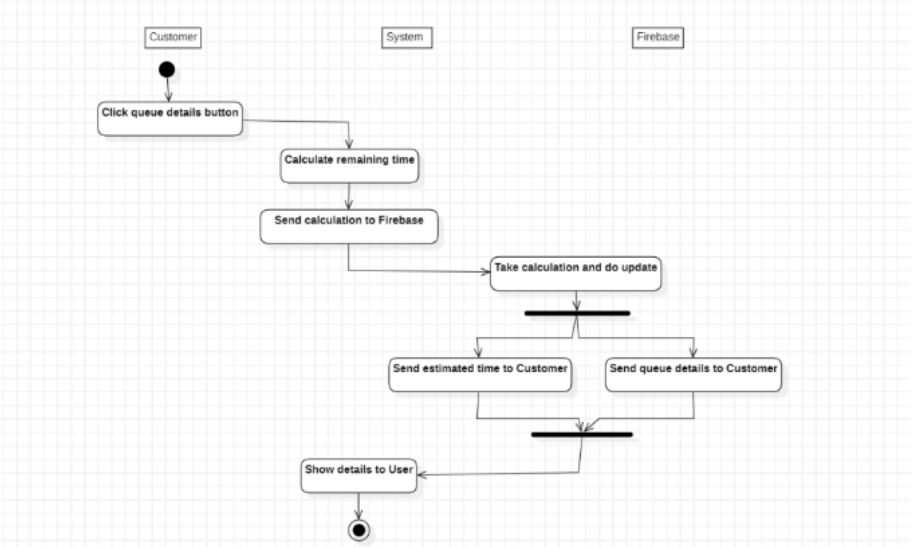



*Figure 9: Activity diagram of view current queue and estimated time*

4.2.2 Sequence Diagrams

Here, we display sequence diagrams of our project's critical use cases. Also, the inclusion and extension use cases were not separated where they are related to only one use case.

Figure 10 shows the sequence diagram of the "View nearest branch list" use case which visualises the interactions between the customer and menu screen, controller, Firebase, and confirmation screen within the system. First, the system will request to open GPS from the customer if it is closed when the "Enter Queue" button is clicked. Then, the system will communicate with Firebase in order to get the nearest branches' information. The system will calculate distances and show the list and show the list to the customer.

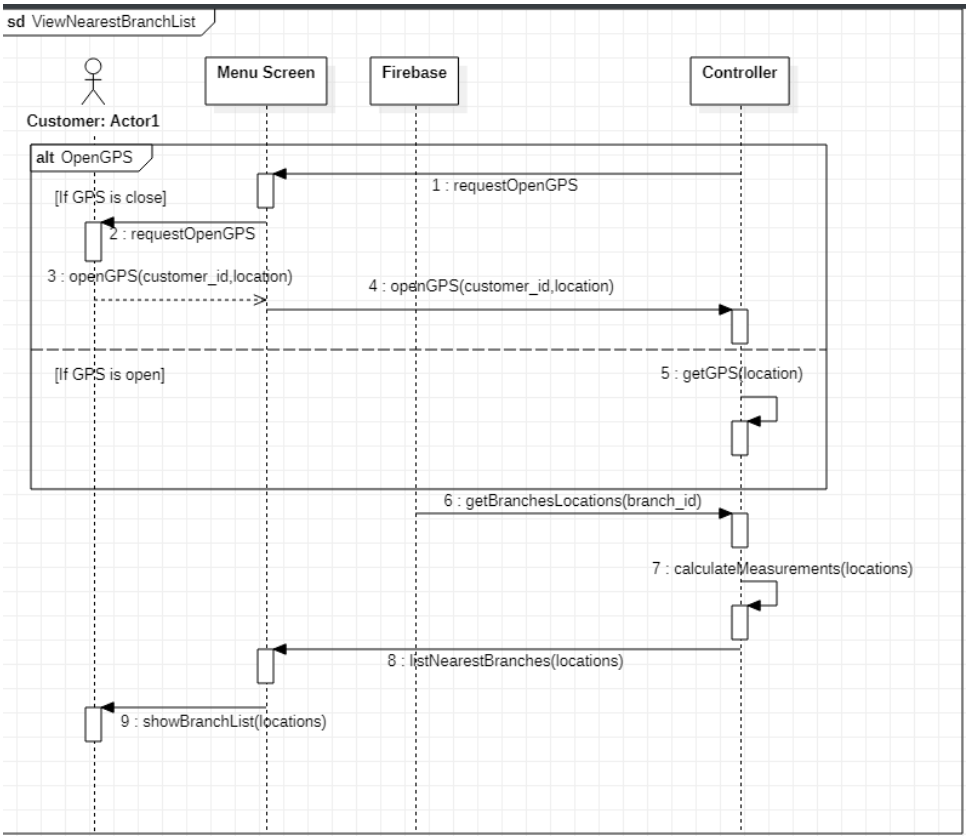



*Figure 10: Sequence diagram of View nearest branch list*

Figure 11 shows the sequence diagram of the "Select Transactions" use case which visualises the interactions between the customer and transaction screen, controller, Firebase, transaction object, customer object, and confirmation screen within the system. The customer must have clicked the "Enter Queue" button and selected a branch beforehand. Here, the customer must click on the transaction s/he wants to achieve. Then, the system will save the transaction selection with customer information in Firebase.

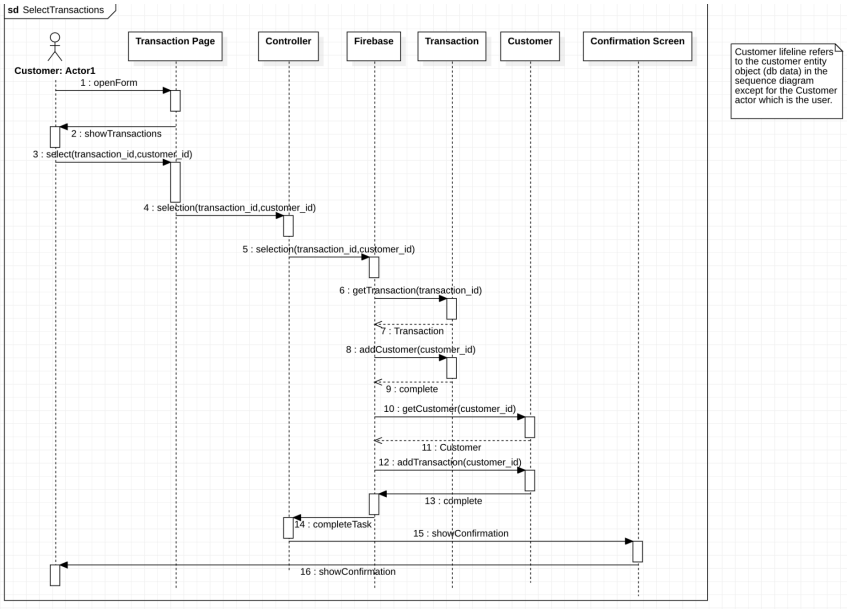



*Figure 11: Sequence diagram of Select Transactions*

Figure 12 shows the sequence diagram of the "Enter the Queue" use case which visualises the interactions between the customer and queue screen, queue controller, Firebase, queue, and confirmation screen within the system. Here, to enter the queue, the customer must have clicked the "Enter Queue" button and saw the branch list. Then, the customer must select a transaction as shown in the previous diagram. The system will add the customer to the queue by changing information about the queue and customer in Firebase.

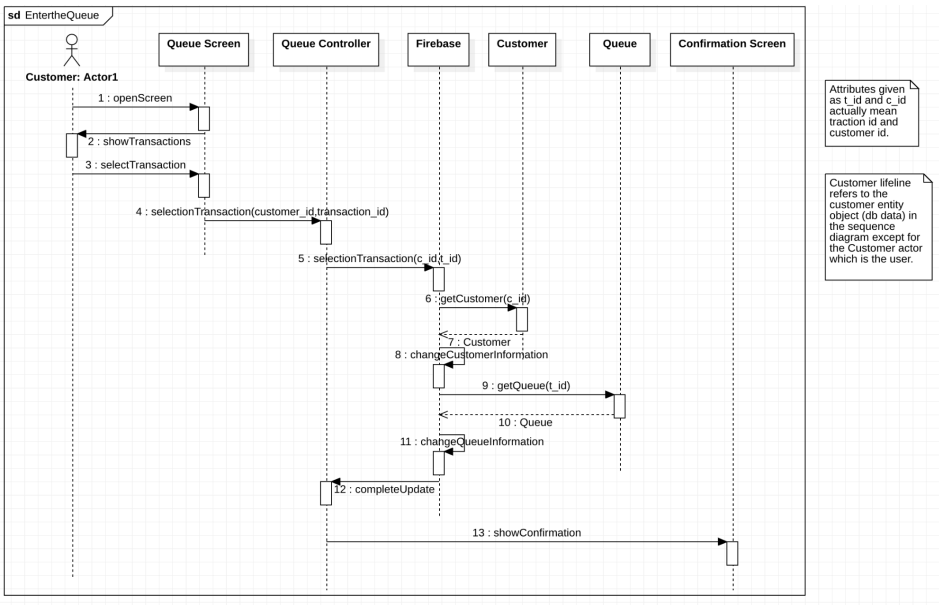



*Figure 12: Sequence diagram of Enter the Queue*

Figure 13 shows the sequence diagram of the "View current queue and estimated time" use case which visualises the interactions between the customer and queue screen, controller, Firebase, within the system. Here, after the customer has entered the queue, the customer must open the active queue screen in order to see the queue information. The system will calculate the remaining time continuously and change the queue details dynamically . This way, the customer will be able to observe up-to-date queue information.

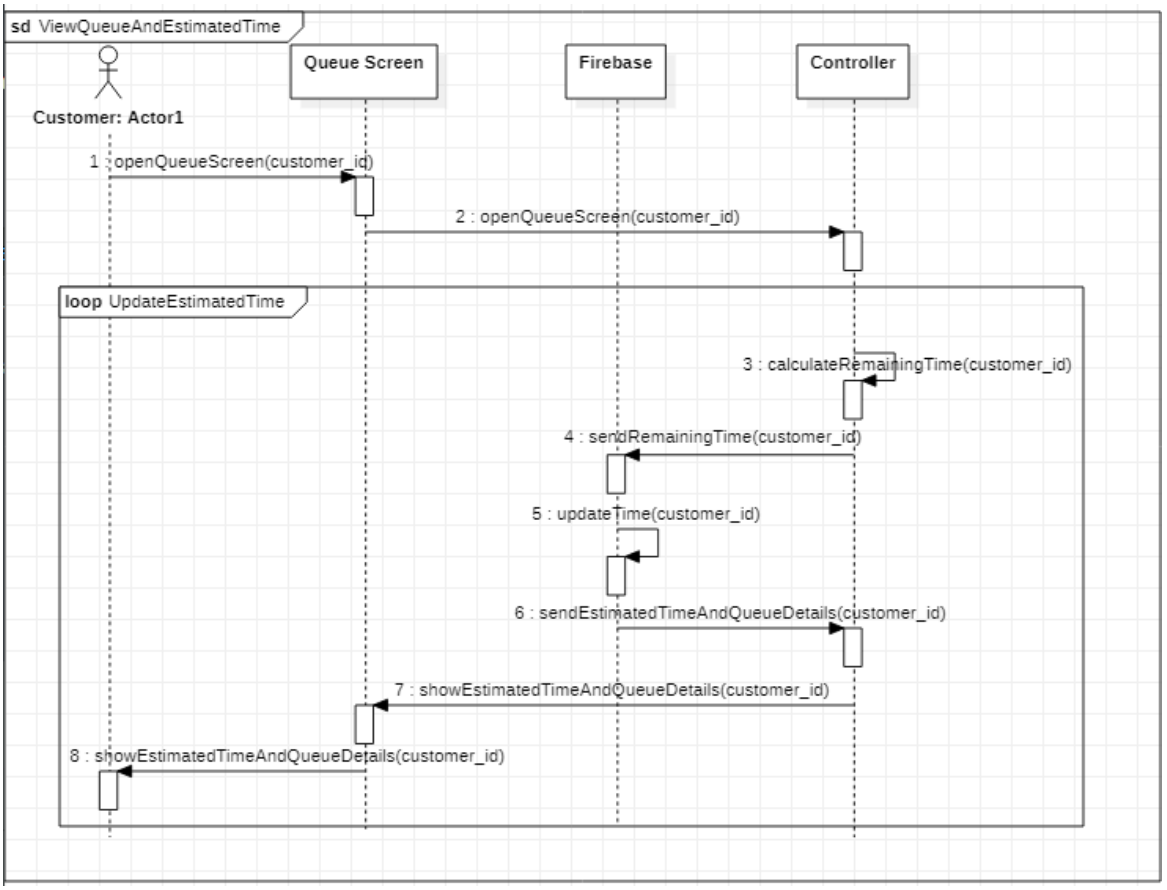



*Figure 13: Sequence diagram of View current  queue and estimated time*

Figure 14 shows the sequence diagram of the "Leave the Queue" use case which visualises the interactions between the customer and queue screen, queue controller, Firebase, queue, and confirmation screen within the system. Request means that a leave request is generated when a customer clicks the leave queue button. Here, to leave the queue, the customer must click the leave button, then the leave request is taken by the system. The system will apply this request by updating the information of the queue and customer in the Firebase.

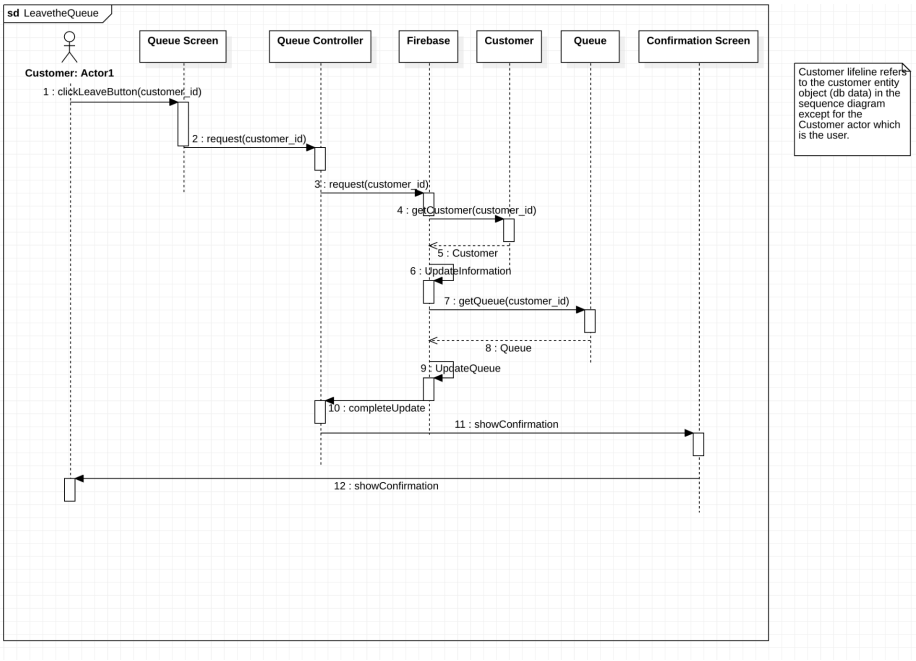



*Figure 14: Sequence diagram of Leave the Queue*

Figure 15 shows the sequence diagram of the "Serve Customer" use case which visualises the interactions between the employee and employee screen, controller, Firebase, and confirmation screen within the system. Here, the employee must click the queue details button to look at the queue information. The system will display the list of operations. Then, if the transaction ended favourably, the employee must select the "complete" button for the successful mark to continue with the line. However, if the transaction did not end successfully, the employee must select the "cancel" for the unsuccessful mark to continue with the line. The system will save this information and send it to the Firebase in order to use it in performance calculations.

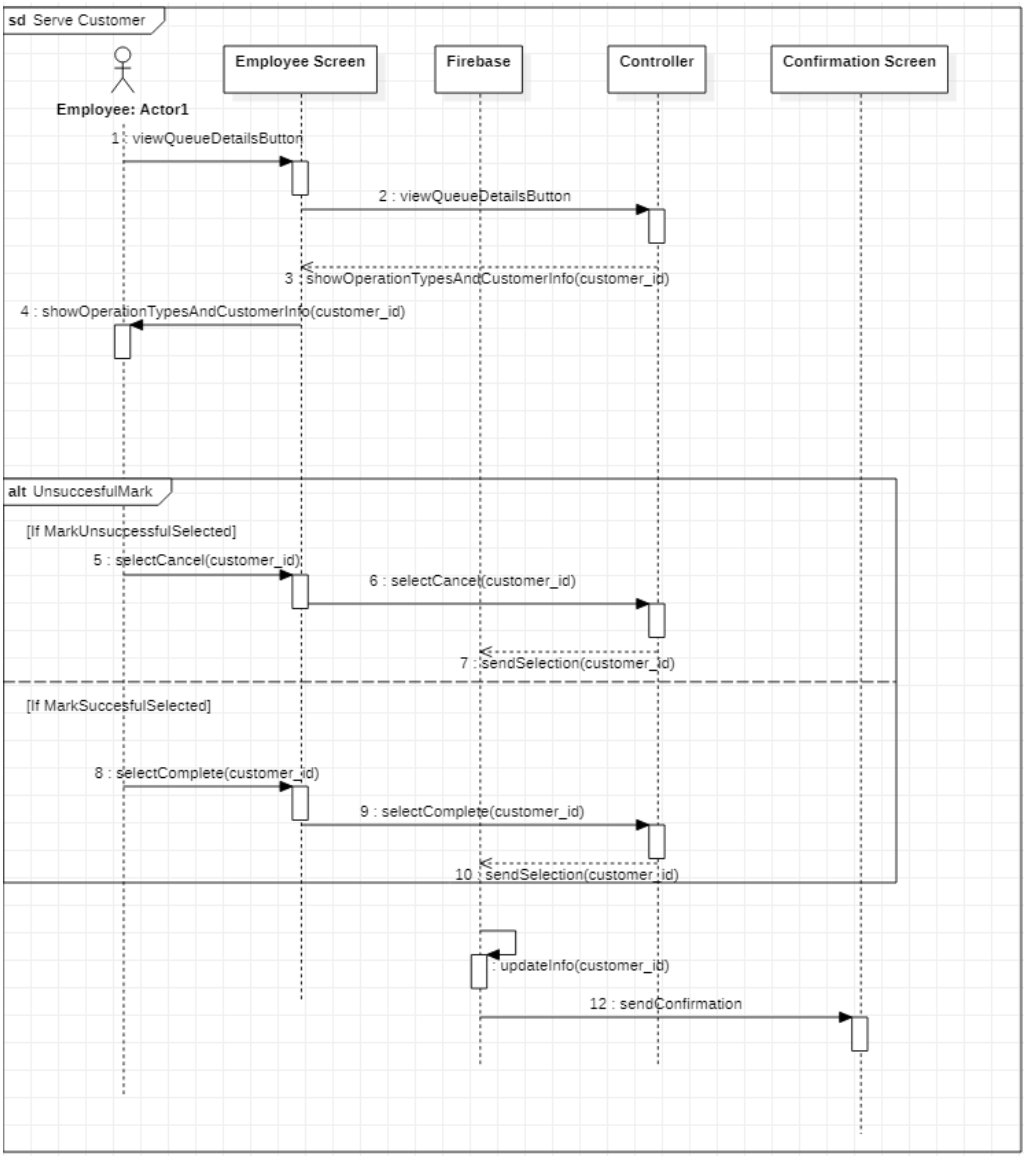



*Figure 15: Sequence diagram of Serve Customer*

### 4.2.3 Data Flow Diagrams

**Context Level:**

In Figure XXXXX, our project's context level DFD which represents the overview of the organisational system is displayed.We have 5 external entities which are Employee, Customer,,Google Firebase, Location Services and ProjectAdmin and a process which is our virtual queue system. Inputs and outputs ofGoogle Firebase are the same

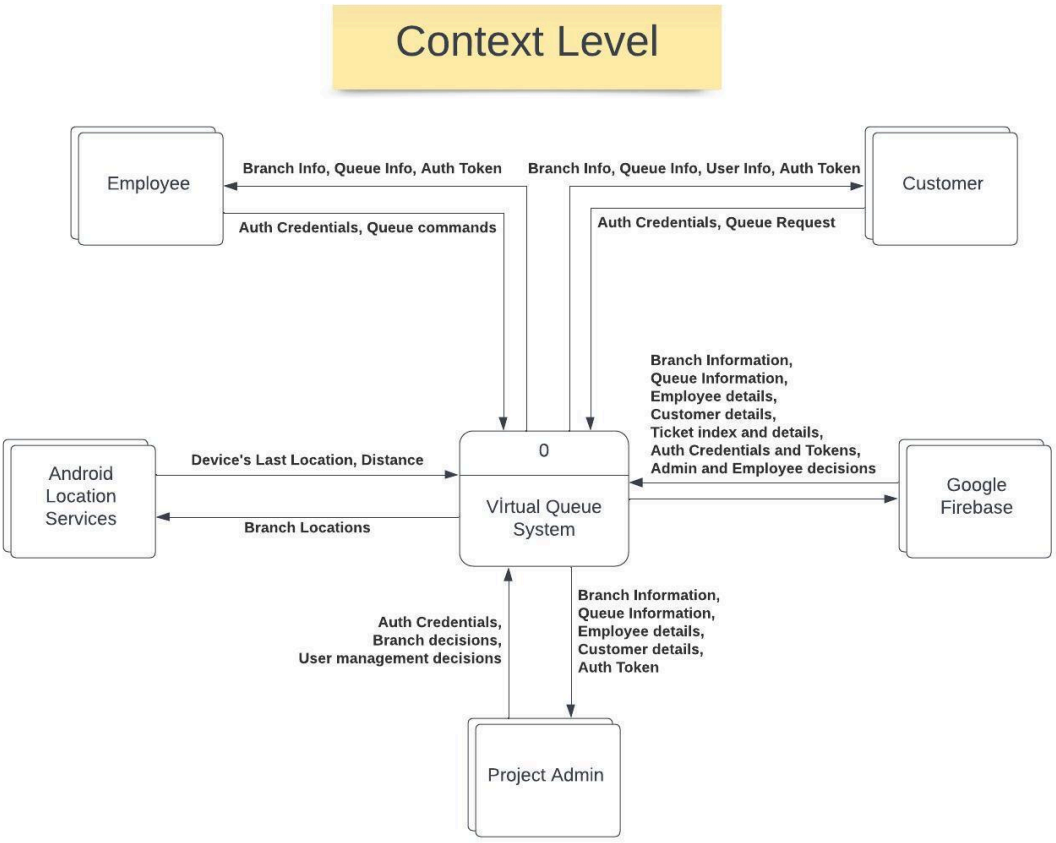



*Figure 16: Context-Level DFD*

**Level 0:**

In Figure XXXXXX, our project's Level-0 level DFD which represents the system's major processes at a high level of abstraction is displayed.We have 4 processes which are Login/Register, Enter Queue, Serve Customer, Manage Branch and data flows between them and external entities.

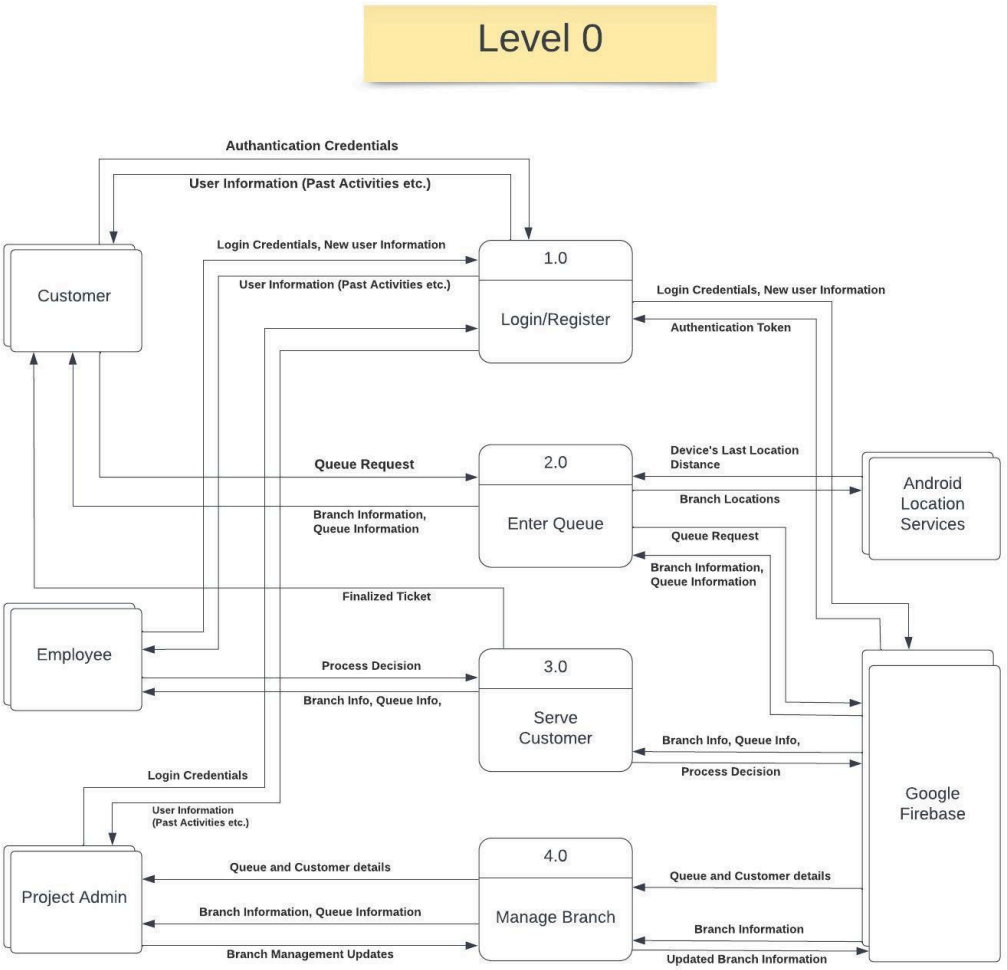



*Figure 17: Level zero DFD*

**Level 1:**

In Figure XXXX, our project's Level-1 level DFD which represents the results from decomposition of the Level-0 diagram is displayed.We have the sub-processes of the processes in the Level-0 diagram.Also in Level-1 diagrams, we combined some data in order to simplify diagrams.

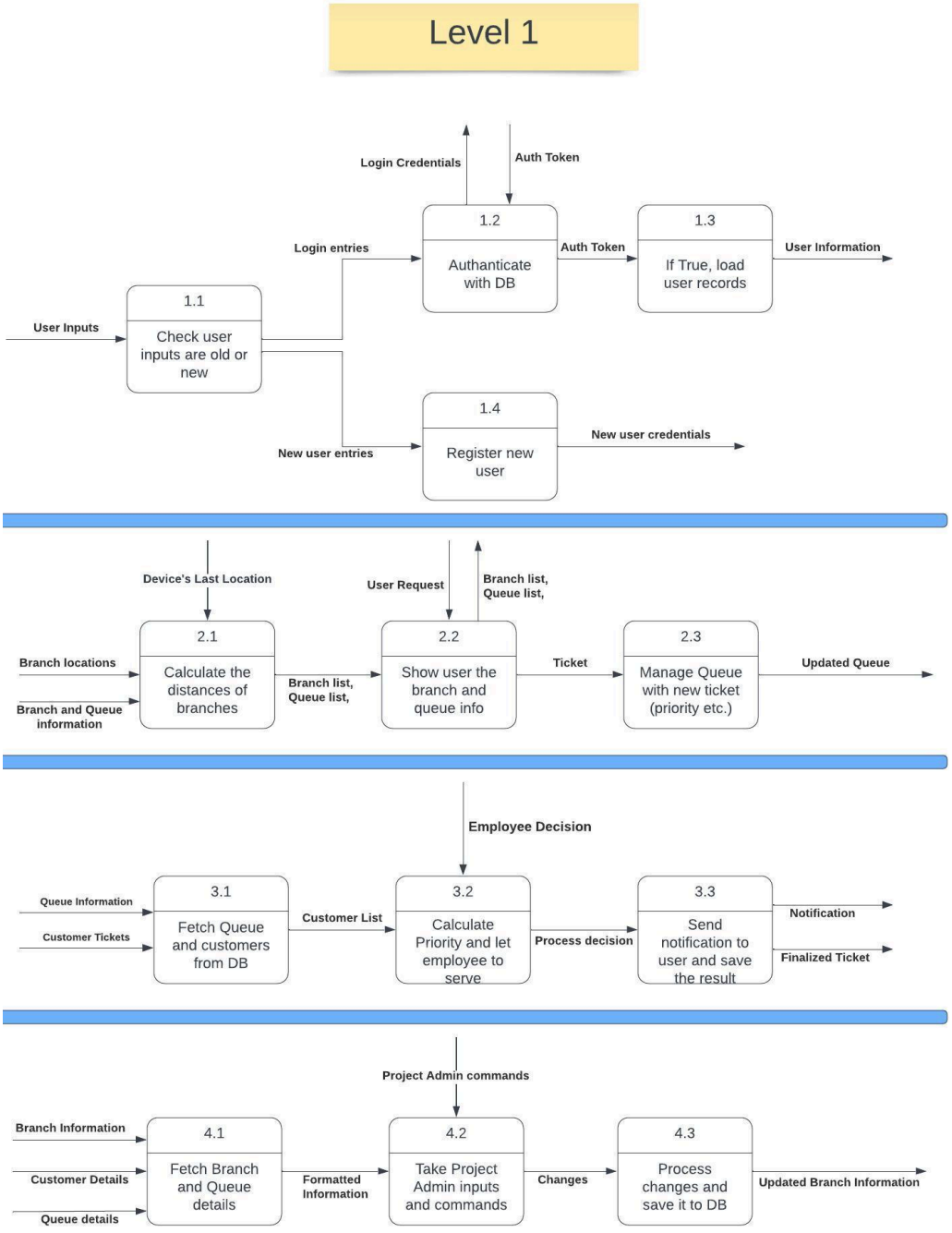



*Figure 18: Level one DFD*

## 4.3 Development View

*The development view shows how the software is decomposed for development.*

### 4.3.1 Component Diagram

According to the class diagram, we checked each class' methods and attributes. For each class, what kind of attribute it stores and which class it needs to access for accomplishing its methods are considered. To set required/provided relations, one component needs to

require other component's provided service. By looking at their needs, these relations are set up in the given figures.

In Figure 16, we did not include User as a component, since it is a superclass but the application mainly focuses on its subclasses which are Admin, Employee and Customer.

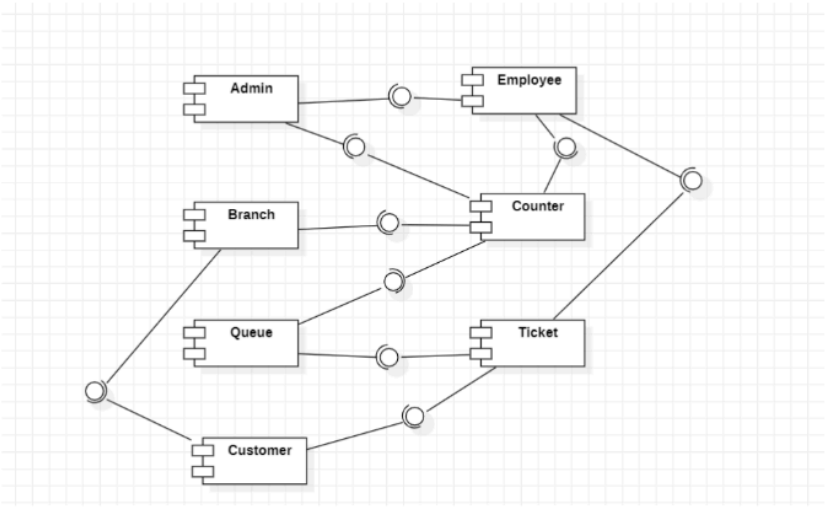



*Figure 19: Component diagram based on classes*

In Figure 17, we provide a different component diagram which shows the relation between our application's screens, admin panel and simulation. Since admin panel and simulation is separated from the mobile application, and they are not a class to store in the database, we did not add them as a component in the Figure 16. Even though they are not a separate class, they accomplish reading and writing operations from the Firebase. They also show the results as a screen on admin panel created for the web. Therefore, we added the Admin Panel and Simulation in this component diagram.

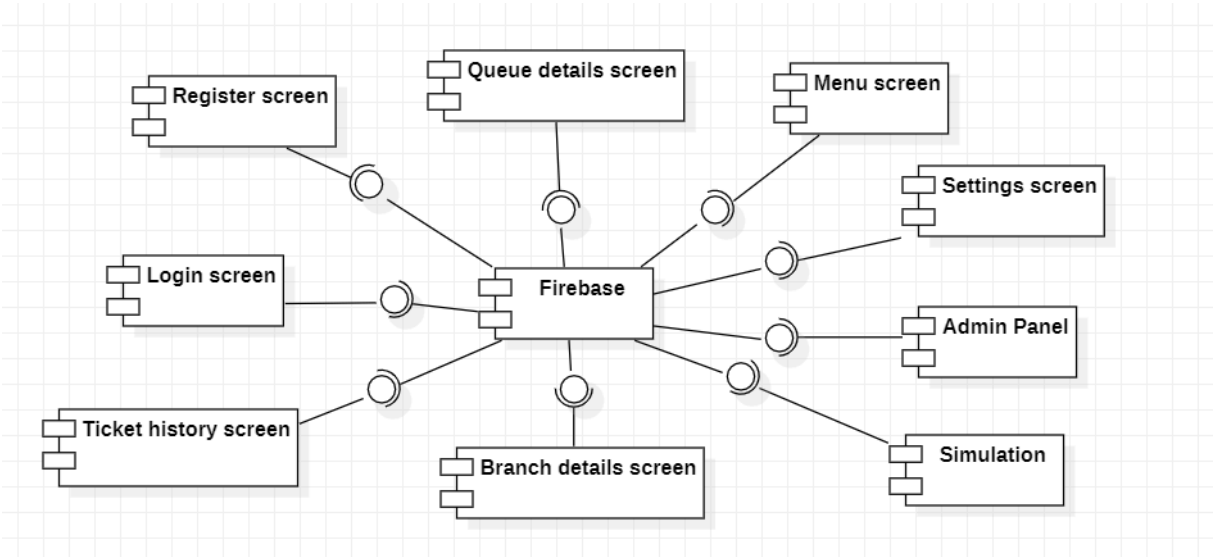



*Figure 20: Component diagram based on screens*

## 4.4 Physical View

*The physical view shows the mapping of software onto hardware.*

### 4.4.1 Deployment Diagram

Since we are using Google Firebase as our database in our project, we need to show the connection with Firebase with the Application.

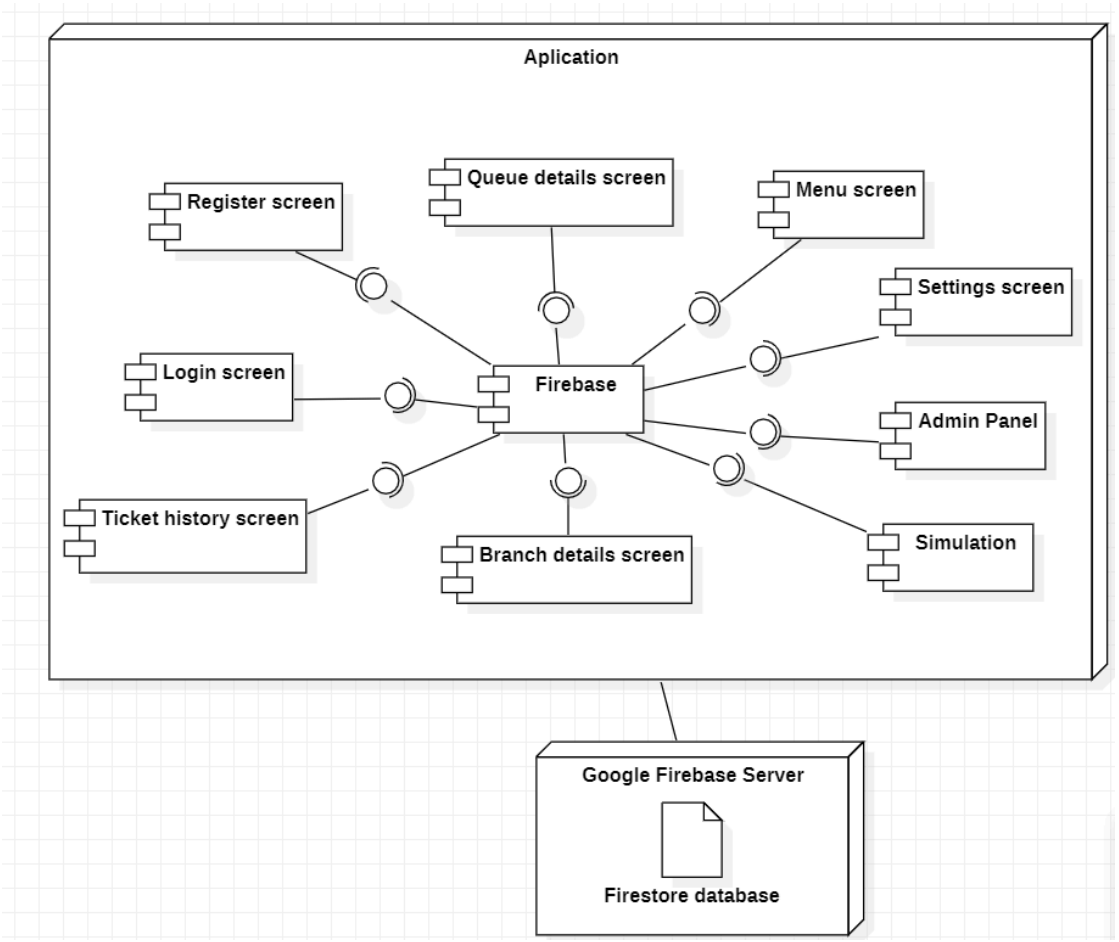



*Figure 21: Deployment diagram*

# 5 Software Implementation

*List of programming languages, frameworks, components, libraries, etc. used with their brief explanations.*

# 6 Software Testing

In the Unit testing part, we tested editing customer details in the Admin Panel, M/M/1 verification in simulation, M/M/C verification in simulation by using Pytest and entering the queue in a mobile application by using JUnit. In total we conducted 14 unit tests.

In the Integration testing part, we tested viewing all branches in customer side, viewing the current queue and estimated time in customer side, viewing the current queue in employee side, viewing all branches in admin side,viewing all customers in admin side, assigning employees in admin side, adding new employee in admin side. In total we conducted 14 unit tests manually from the user side.

In the System testing part, we tested entering the queue and leaving the queue on the mobile application side manually. On the admin panel side, we tested editing customer details and adding a new employee manually.

In the Performance testing part, we tested running MM1 simulation model performance based on total customer number with the same priority and with different priorities by giving parameters manually.

In the Usability testing part, we tested serving the customer on the employee side, leaving the queue on the customer side, editing customer details in the admin side, assigning employees on the admin side,and entering the queue on the customer side. They are done by five friends of ours.

We completed all the tests as we planned clearly while preparing the Test Plan report. However, not all the tests have passed. We took the notes, and we updated our missing parts in each part of our software product.

## 6.1 Unit Testing

### 6.1.1 Test Procedure

For the Admin Panel and Simulation tests, we decided to apply Pytest. On the other hand, we used JUnit to conduct mobile application tests. We first identified the test cases and then gave these cases to our tester. Since the test cases have descriptions, detailed steps' explanation, and test data, our tester could easily understand and apply the required test by looking at these test case values.

We specified the unit tests we are planning to in Test Plan report which are the followings:

1. Editing customer details in Admin Panel
2. Entering the queue in Mobile Application
3. M/M/1 Verification in Simulation
4. M/M/C Verification in Simulation

Our tester for the Unit test was İrem.

We provide two example test codes and explanations of our unit tests below.

To show how we conduct our Pytest scripts, we first chose the "Check Edit Customer with Valid Data of priority in Admin Side" case which is shown as test ID 1 in 6.1.2. part.

Written script for the test ID 1:

```python
def test_customer_edit_priority():
    uid="TE8qNNLVkhRrfLn2huYTys8FaW82"
    customersref = db_config.db.collection('Customers')
    query = customersref.where("uid", "==", uid).stream() # filtering according to the passed uid input
    for doc in query:
        customer = doc.to_dict()
    new_priority=3
    customer_edit(uid,customer['name'],customer['surname'],new_priority,customer['reg_date'],customer['email'])

    customersref = db_config.db.collection('Customers')
    query = customersref.where("uid", "==", uid).stream()
    for doc in query:
        updated_customer = doc.to_dict()
        assert updated_customer['priority'] == new_priority
```

*Figure 22: Test script for test ID 1*

We are taking a customer's info from the database with the customer's uid to update his priority to 3 from 1. We enter a new priority for the customer. After that, we connect to the database again and take the updated info of the customer to check if the priority is updated. We use 'assert', which is a keyword of Pytest, to test if the database value is equal to the one we entered.

After the test ID 1, we selected the test which has ID 12 to explain:

```python
import bank_simulation

'''def test_sim_mm1_utilisation():
    mql,equation,u= bank_simulation.start(1)
    assert u <= 1'''
def test_sim_mm1_mql():
    mql,equation,u= bank_simulation.start(1)
    dif=equation-mql
    discrepancy=(abs(dif)/equation)*100
    assert discrepancy <= 5
```

*Figure 23: Test script for test ID 12*

We run the simulation code with the data from our database, Firestore, with one server. We tested our code with Pytest using the 'assert' keyword. First, we take the calculated mean queue length result and the wanted analytic formula result as output from simulation. Then, we calculated discrepancy for M/M/1 mean queue length result and the analytic formula result to check if it is under the %5 which is the acceptable limit.

### 6.1.2 Test cases

| ***ID*** | ***Description*** | ***Steps*** | ***Test Data*** | ***Pre-condition*** | ***Expected Output*** | ***Passed/ Failed*** |
|---|---|---|---|---|---|---|
| 1 | Check Edit Customer with Valid Data of priority in Admin Side | 1.Call customer_edit function in test_customer_edit file with new priority | priority=3 | priority value should be assigned previously | Successful realtime update operation in db | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | 2. Run the Pytest<br>3.Check the result | | | | |
| 2 | Check Edit Customer with Valid Data of name,surname and email in Admin Side | 1.Call customer_edit function in test_customer_edit file with new name,surname and email<br>2. Run the Pytest<br>3.Check the result | name="Test Name" surname="TestSurame" email="test_email@gmail.com" | The customer must be exist in the db | Successful realtime update operation in db | Passed |
| 3 | Check Edit Customer with Invalid Data of email in Admin Side | 1.Call customer_edit function in test_customer_edit file with invalid email<br>2. Run the Pytest<br>3.Check the result | email="1433" | The customer must be exist in the db | Give an error and do not change the email | Passed |
| 4 | Check Edit Customer with Invalid Data of Birth date in Admin Side | 1.Call customer_edit function in test_customer_edit file with invalid date<br>2. Run the Pytest<br>3.Check the result | birth date="2000" | The customer must be exist in the db | Give an error and do not change the birth date | Passed |
| 5 | Check Edit Customer with Invalid Data of name, surname, priority and reg date in Admin Side | 1.Call customer_edit function in test_customer_edit file and update the customer with empty name,surname,priority and date<br>2. Run the Pytest | name=" " surname=" " priority=" " reg date= " " | The customer must be exist in the db | Give an error and do not change the values | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | 3.Check the result | | | | |
| 6 | Check Edit Customer with Invalid Data of name,surname, email and priority in Admin Side | 1.Call customer_edit function in test_customer_edit file and update the customer with empty name,surname,email and priority<br>2. Run the Pytest<br>3.Check the result | name=" "<br>surname=" "<br>email=" "<br>priority=" " | The customer must be exist in the db | Give an error and do not change the values | Passed |
| 7 | Check Edit Customer with Invalid Data of name, surname and priority in Admin Side | 1.Call customer_edit function in test_customer_edit file and update the customer with empty name,surname and priority<br>2. Run the Pytest<br>3.Check the result | name=" "<br>surname=" "<br>priority=" " | The customer must be exist in the db | Give an error and do not change the values | Failed |
| 8 | Check Delete Customer with Valid Data in Admin Side | 1.Call delete_customer function in test_delete_customer file and check the if user is in the db<br>2. Run the Pytest<br>3.Check the result | userUid="KL5UN2EIXYaYjap1TQpVnjXQljv2" | The customer must be exist in the db | Give an error for UserNotFound, Successful realtime delete operation in db | Passed |
| 9 | Entering the queue with Valid Data in Customer Side | 1.Call checkQueueWithCorrectDatafunction in EnterQueueTest file and check the if | priority=3<br><br>processType="Payments"<br><br>userUid="KL5UN2EIXYaY | The customer must be exist in the db | User with wanted info should be in the queue in db | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | user is in the queue in db<br>2. Run the JUnit<br>3.Check the result | jap1TQpVnjXQljv2" | | | |
| 10 | Entering the queue with Invalid Data in Customer Side | 1.Call checkQueueWithInvalidUid in EnterQueueTest file and check the if user is in the queue in db<br>2. Run the JUnit<br>3.Check the result | priority=3 processType="Payments" userUid="test" | | User with wrong uid should not be ib the queue in db and gives an error | Passed |
| 11 | M/M/1 Utilisation Verification in Analytical Model Simulation | 1.Call the test code by calling the simulation code with 1 server option<br>2. Run the Pytest<br>3.Check the result | retrieved from Firestore database | | Utilisation result should be close to 1. | Passed |
| 12 | M/M/1 MQL Verification in Analytical Model Simulation | 1.Call the test code by calling the simulation code with 1 server option<br>2. Run the Pytest<br>3.Check the result | retrieved from Firestore database | | Discrepancy between MQL result and ro/-1-ro result should be less or equal to 5%. | Passed |
| 13 | M/M/C Verification in Mathematical Model Simulation | 1.Call the test code by calling the simulation code with multiple server option<br>2. Run the Pytest<br>3.Check the result | customer num:50,arrival_rate:3, service_rate:4,server number:3 | | Utilisation result should be close to 1. | Passed |
| 14 | M/M/1 Verification in Mathematic | 1.Call the test code by calling the simulation code with | customer num:50,arrival_rate:3, | | MQL result from simulation and MQL in the system formula's result | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | al Model Simulation | multiple server option<br>2. Run the Pytest<br>3.Check mean queue length | service_rate: 4,server number:3 | | should be nearly equal. | |

*Table 3: Test case table for Unit testing*

### 6.1.3 Test Results

All the conducted tests on the Admin Panel except test ID 7 which is "Check Edit Customer with Invalid Data of name, surname and priority in Admin Side" are passed. We checked our implementation and detected that there is error checking for empty parameter for "reg date" and "email" inputs. However, there is no error checking for the priority input. After detecting this error, we added error check for "priority" input as well. Problem is solved.

All the conducted tests on Simulation are passed.

All the conducted tests on Mobile Application are passed.

### *6.1.4 Test Log*

We run both test ID 1 and 2 consequently and we got the following output screen as below:

```
PS C:\Users\Irem\Desktop\unit_testing\admin> pytest
============================================================ test session starts ============================================================
platform win32 -- Python 3.11.2, pytest-7.3.1, pluggy-1.0.0
rootdir: C:\Users\Irem\Desktop\unit_testing\admin
collected 2 items

test_customer_edit.py ..                                                                                                           [100%]

============================================================ warnings summary ============================================================
test_customer_edit.py::test_customer_edit_priority
test_customer_edit.py::test_customer_edit_priority
  C:\Users\Irem\AppData\Local\Programs\Python\Python311\Lib\site-packages\urllib3\util\ssl_.py:250: DeprecationWarning: ssl.PROTOCOL_TLS is deprecated
    context = SSLContext(ssl_version or ssl.PROTOCOL_SSLv23)

test_customer_edit.py::test_customer_edit_priority
test_customer_edit.py::test_customer_edit_priority
  C:\Users\Irem\AppData\Local\Programs\Python\Python311\Lib\site-packages\urllib3\connection.py:356: DeprecationWarning: ssl.match_hostname() is deprec
ated
    match_hostname(cert, asserted_hostname)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================================= 2 passed, 4 warnings in 5.49s =========================================================
PS C:\Users\Irem\Desktop\unit_testing\admin>
```

*Figure 24: Test result screen for both test ID 1, and test ID 2*

Secondly, we selected the "MM1 MQL Verification in Analytical Model Simulation".

Output screen for the test ID 12:

```
OUTPUT  TERMINAL  DEBUG CONSOLE  PROBLEMS                                                                                    Code

============================================================================ test session starts ============================================================================
platform win32 -- Python 3.11.2, pytest-7.3.1, pluggy-1.0.0
rootdir: C:\Users\Irem\Desktop\unit_testing\simulationAnalitic
collected 1 item

test_sim_mm1.py .                                                                                                                                                  [100%]

============================================================================ 1 passed in 4781.84s (1:19:41) =============================================================
PS C:\Users\Irem\Desktop\unit_testing\simulationAnalitic>
```

*Figure 25: Test script for test ID 12*

In Figure 25, it is clearly shown that, test has passed successfully.

6.2 Integration Testing

6.2.1 Test Procedure

We did not use any additional tools or techniques for the integration tests. We tested them manually. We first identified the test cases and then gave these cases to our tester. Since the test cases have descriptions, detailed step's explanations, and test data, our tester could easily understand and apply the required test by looking at these test case values.

We specified the integration tests we are planning to in Test Plan report which are the followings:

1. Viewing all branches in customer side
2. Viewing the current queue and estimated time in customer side
3. Viewing the current queue in employee side
4. Viewing all branches in admin side
5. Viewing all customers in admin side
6. Assigning employees in admin side
7. Add new employee in admin side

Our tester for the Integration tests was Eylül.

6.2.2 Test cases

| *ID* | *Description* | *Steps* | *Test Data* | *Pre-condition* | *Expected Output* | *Passed/ Failed* |
|---|---|---|---|---|---|---|
| 1 | View All Branches in Customer Side | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View Branch | Email="emrecan2012@yahoo.com"<br><br>Password: "123456" | The customer and branches must be exist in the db | Kalkanlı Branch 1310 meters<br>Nicosia Branch 27856 meters<br>Kyrenia Branch 30329 meters | Passed |
| 2 | View Current Queue and Estimated Time for position checking in Customer Side | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View Branch<br>5. Select a Branch<br>6. Enter a queue | Email="emrecan2012@yahoo.com"<br><br>Password: "123456" | The customer and branches must be exist in the db | Successful queue position and estimated time view | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| 3 | View Current Queue and Estimated Time for dynamic position changing in Customer Side | 1.Enter the e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View Branch<br>5. Select a Branch<br>6. Enter a queue<br>7. Wait a while to see someone leave<br>8.Check active queue page again | Email="emrecan2012@yahoo.com"<br><br>Password: "123456" | The customer, queues, and branches must be exist in the db | Successful dynamic change in waiting time and position observation | Passed |
| 4 | View Current Queue and Estimated Time for dynamic waiting time and positin changing in Customer Side | 1.Enter the e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View Branch<br>5. Select a Branch<br>6. Enter a queue<br>7. Wait a while to see someone leave<br>8.Check active queue page again<br>9.Check db | Email="emrecan2012@yahoo.com"<br><br>Password: "123456" | The customer, queues, and branches must be exist in the db | Successful dynamic change in waiting time and position in db | Passed |
| 5 | View Current Queue in Employee Side | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4.Click Manage Queue | Email="emrecan2012@yahoo.com"<br><br>Password: "123456" | The employee, his queue, and his registered branch must be exist in the db | Successful active customer and queue information observation | Passed |
| 6 | View All Branches in Admin Side | 1.Enter e-mail | Email="cidil@metu.edu.tr" | cidil@metu.edu.tr | Successful branch list, employee | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | 2.Enter password<br>3.Click Login<br>4.Click View List<br>5.Click View Branches | Password ="123456" | should be in the db | list,queue details and add new employee buttons view | |
| 7 | View All Customers in Admin Side | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View List<br>5.Click View Customers | Email="cidil@metu.edu.tr" Password ="123456" | cidil@metu.edu.tr should be in the db | Successful add new customer button, customer list including name, surname, priority, email and age information view | Passed |
| 8 | View All Customers for opening the new customer form in Admin Side | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View List<br>5.Click View Customers<br>6. Click Add New Customer | Email="cidil@metu.edu.tr" Password ="123456" | cidil@metu.edu.tr should be in the db | Successful empty new customer form appearing | Passed |
| 9 | Assigning Employees in Admin Side with Valid Data of email, password and branch name | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4. Click View List<br>5.Click View Employees<br>6.Click Edit Button for one Employee<br>7. Change Branch Name<br>8.Click Submit<br>9. Check database | Email=cidil@metu.edu.tr Password ="123456" Branch name="Kyrenia" | cidil@metu.edu.tr should be in the db | Successful employee form appearing, dynamic branch name change in db | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| 10 | Assigning Employees in Admin Side with Valid Data of email,password and Invalid Data of empty branch name | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4. Click View List<br>5.Click View Employees<br>6.Click Edit Button for one Employee<br>7. Change Branch Name<br>8.Click Submit | Email="cidil@metu.edu.tr" Password ="123456" Branch name=" " | cidil@metu.edu.tr should be in the db | Error Message | Passed |
| 11 | Assigning Employees in Admin Side with Valid Data of email,password and Invalid Data of non existing branch name | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4. Click View List<br>5.Click View Employees<br>6.Click Edit Button for one Employee<br>7. Change Branch Name<br>8.Click Submit | Email="cidil@metu.edu.tr" Password ="123456" Branch name=" Mersin " | cidil@metu.edu.tr should be in the db | Error Message | Failed |
| 12 | Add New Employee in Admin Side with Valid Data of email and password | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View List<br>5.Click View Branches<br>6. Click Add New Employee in one Branch | Email="cidil@metu.edu.tr" Password ="123456" | cidil@metu.edu.tr should be in the db | Successful employee form appearing | Passed |
| 13 | Add New Employee in Admin Side with Valid | 1.Enter e-mail<br>2.Enter password | Email="cidil@metu.edu.tr" | cidil@metu.edu.tr should be in the db | Error message | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | Data of email, password, name, surname, birth date, and Invalid Data of email | 3.Click Login<br>4.Click View List<br>5.Click View Branches<br>6. Click Add New Employee in one Branch | Password ="123456" Name="E ren" Surname= "Cihan" Email=" " Birth Date="18-01-1990" | | | |
| 14 | Add New Employee in Admin Side with Valid Data of email,password,name,surname,email, and birth date | 1.Enter e-mail<br>2.Enter password<br>3.Click Login<br>4.Click View List<br>5.Click View Branches<br>6. Click Add New Employee in one Branch<br>7. Enter the values<br>8. Click Submit<br>9. Check db | Email="cidil@metu.edu.tr" Password ="123456" Name="E ren" Surname= "Cihan" Email=" erencihan @employee.com" Birth Date="01/18/1990" | cidil@metu.edu.tr should be in the db | Successful new employee addition for the selected Branch, dynamic update in db | Passed |

*Table 4: Test case table for Integration testing*

6.2.3 Test Results

All the conducted tests for the Employee side are passed.

All the conducted tests for Admin side except the test ID 11 are passed. We did not see any error message when we tried to update the branch name with a name which is not in the database. It just takes the selected employee from his/her current branch and makes his/her branch name information empty. We added an error check and error message there and did the test again. Problem is solved.

All the conducted tests for Customer side are passed.

*6.2.4 Test Log*

We put Test ID 6's and Test ID 14' results as examples of our conducted tests.

For the Test ID 6 :


*Figure 26: Test result screen for test ID 6*

All the branch lists including their employee list button, queue details button, and add new employees button appeared.

For the Test ID 14:

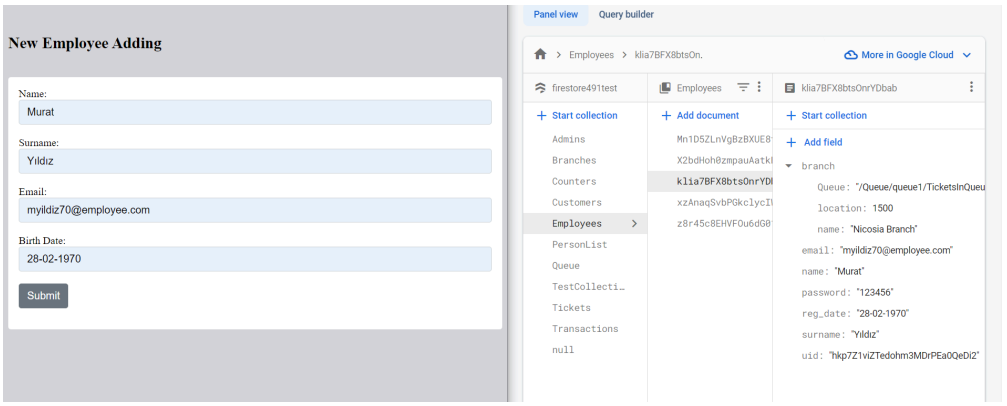



*Figure 27: Test result screen for test ID 14*

New Employee for the Nicosia Branch is added in the db successfully.

## 6.3 System Testing

### 6.3.1 Test Procedure

We did not use any additional tools or techniques for the System tests. We tested them manually. We first identified the test cases and then gave these cases to our tester. As the system testing focuses on evaluating the complete and integrated system to ensure that it meets the specified requirements and functions as intended, we needed to test our independent systems which are mobile application and admin panel. Thus, we decided to write two system tests for our mobile application and two system tests for our admin panel.

We specified the system tests we are planning to in Test Plan report which are the followings:

1. Entering the queue
2. Leaving the queue
3. Editing customer details
4. Adding a new employee

Our tester for the System testing was İrem.

6.3.2 Test Scenarios

Scenarios we want to test for the "Entering the queue" are the followings:

1- Mert was working as an instructor at Near East University. After his classes were done, his sister called and said that she needed a large amount of cash. Mert was angry because he had given late notice. He should have handled it very fast. He heard before that there is an application for the bank to get in the queue online. He opened the Google Play Store and downloaded the application, then he registered the online queue application. After he logged into the system, he wanted to see if he could go to the Kyrenia Branch or Nicosia Branch to get the job done quicker. That's why he clicked the enter queue button to see which branch was closer and had less people in the queue. Although the queue is shorter in Kyrenia, he also thought about the distance from Kyrenia to her home. It would be more profitable if he went to Nicosia. He got in line for Nicosia Branch by choosing the Withdraw/Deposit Money transaction and got in his car and drove off. He checked from his phone how much time there was till his position in real time. Thanks to the online queuing application, when he arrived there he waited only 2 minutes and left the queue when it was his turn. He had done the job as expected and helped his sister in a timely manner.

Scenarios we want to test for the "Leaving the queue" are the followings:

2- Zeynep had a car model that she liked for a long time. She was finally able to find that car in Nicosia. However, his seller was in a hurry and therefore she needed to get loan approval very quickly. She had a mobile application to get in the line online for the bank already. She logged into the application. She quickly clicked the enter the queue button and chose the Nicosia Branch for a loan application. She checked how much time she had before it was her turn. Her 3-year-old son's teacher called her suddenly. Berk had fallen at school, and she had to take him to the hospital. She had only 10 minutes until her turn. So, she immediately left the queue in the app. When she arrived at the school, she realized that there was no need to be taken to the hospital and opened the mobile application to re-enter the queue. She saw that if she enters the queue again from Nicosia, she will have to wait a long time. She canceled the plan and decided to go tomorrow.

Scenarios we want to test for the "Editing customer details" are the followings:

3- Şükrü was the bank manager for A bank. He got a call from one of his branch managers, Yeliz. She said that there was a customer who logged into the bank system and entered the queue for Nicosia Branch but he did not have an age priority even though he was 70. It was strange. Şükrü told Yeliz that he was going to check the situation and he asked for the customer's information, then hung up. He logged into the Admin Panel for the Bank. He saw the statistics for today on the home page, they had a lot of customers that day. He smiled. Then, he clicked the view

customer list button to see the information of that customer. He opened the information that Yeliz sent him. Later, he realized that the customer entered his birth date "03/04/1963" instead of "03/04/1953" while registering to the app. He clicked the edit button to fix this issue. He updated his birth date and priority. He logged out from the Admin Panel. He called Yeliz and let her know that he corrected the mistake. He thought, now, he deserved a nice cup of coffee.

Scenarios we want to test for the "Adding a new employee " are the followings:

4- İdil was the bank manager for B bank. She interviewed 10 people for employee positions in the Nicosia and Kyrenia Branch of the Bank. After a long election session, she decided to hire 2 of the candidates. She emailed them and let them know that they were accepted. Then, she sent them their employee emails and temporary passwords.Now, it was time to add them to the bank system. She logged into the Admin Panel for the Bank. She clicked on the View Branches button to display all branches of the bank. She clicked the "add new employee" button for Nicosia Branch to add that young girl with no experience. She took a risk by employing her but if no one gave the chance how could she start her career? She entered her name, surname, new email and birth date. It was done. Then, she added the other new employee to the Kyrenia Branch. Her job was done. She logged out from the webpage and went out to have a nice lunch with her friends.

### 6.3.3 Test Results

Conducted tests are passed. Since we tried to test all the possible error cases while developing the mobile app, we did not face problems while testing our mobile application.

### *6.3.4 Test Log*

We conducted all scenarios in this part and put the first scenario and third scenario as examples of our tests.

For the first scenario:

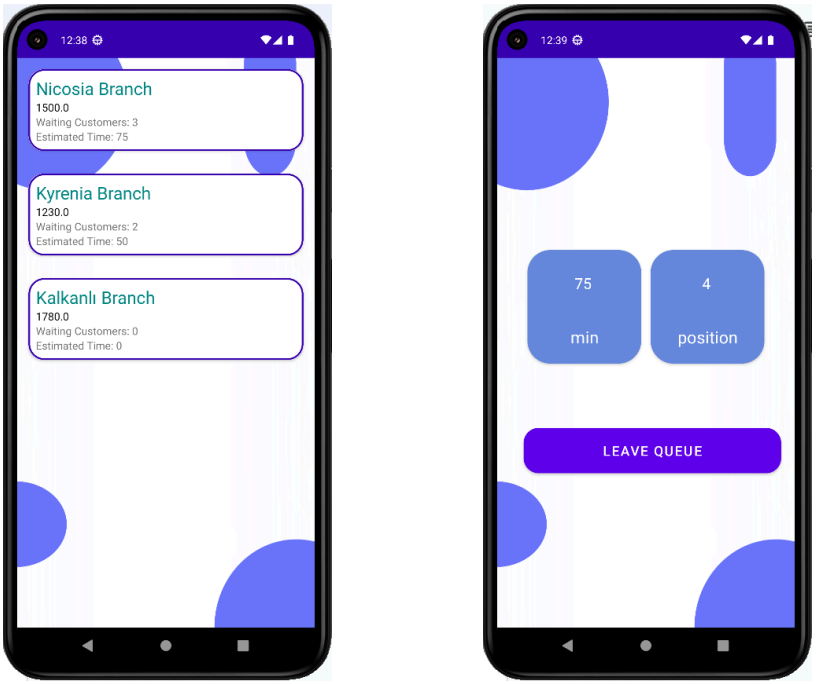


*Figure 28: Test result screen for the first scenario*

Branch List screen shows the available branches. As mentioned in the scenario, even though the Kyrenia Branch's waiting time is less than the Nicosia one, he would like to join the Nicosia Branch.

For the third scenario:

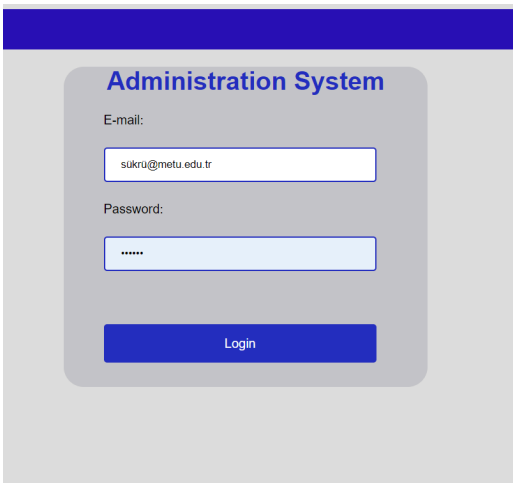



*Figure 29: Log in screen*


| name | surname | priority | email | age |
|---|---|---|---|---|
| Olivia | Jovi | 1 | olivia-jovi@gmail.com | 30 |
| Mehmet | Crawford | 2 | mehmet.crawford@gmail.com | 51 |
| Olivia | Simpson | 3 | olivia-simpson@gmail.com | 25 |
| Beyza | Kobain | 2 | beyza.kobain@gmail.com | 78 |
| Joe | Yilmaz | 3 | joe-yilmaz@gmail.com | 55 |
| Joe | Kobain | 2 | joekobain@gmail.com | 48 |
| Joan | Simpson | 3 | joan*simpson@gmail.com | 30 |
| Joe | Ferah | 2 | joe+ferah@gmail.com | 48 |
| Mehmet | Ferah | 1 | mehmet+ferah@gmail.com | 60 |
| John | Crawford | 2 | john-crawford@gmail.com | 85 |
| Beyza | Lennon | 3 | beyza.lennon@gmail.com | 50 |

*Figure 30: Viewing Customer list*

Here, you can see the login screen and customer list as mentioned in the test.

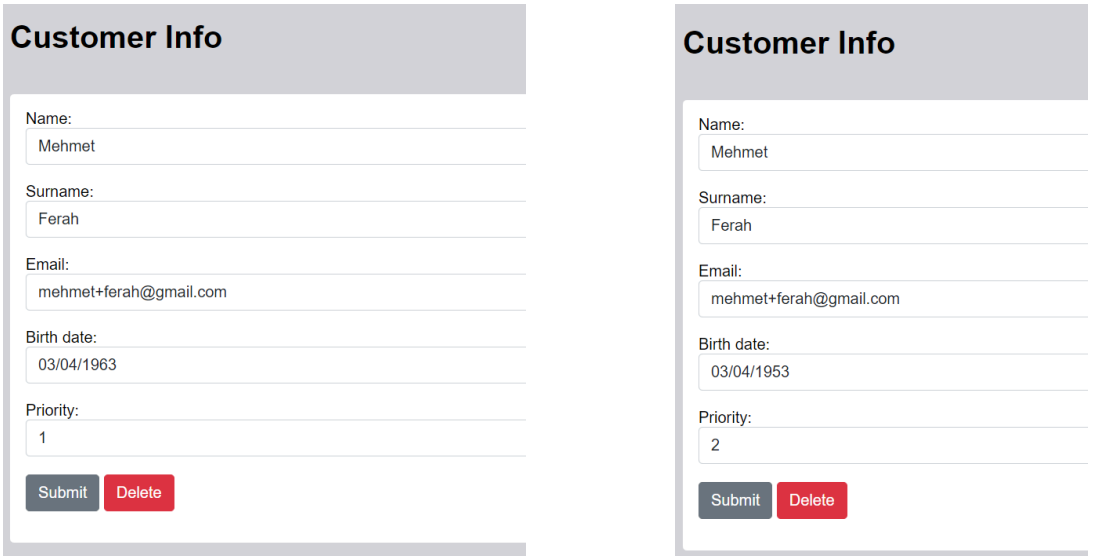



*Figure 31: Edit customer page before and after update*


*Figure 32: Viewing Customer list after update*

In figure 31, the admin has updated the customer details and later, in figure 32, he checked if customer information is updated correctly.

## 6.4 Performance Testing

### 6.4.1 Test Procedure

We did not use any additional tools or techniques for the Performance tests. We tested them with our test script written for the simulation. We first identified the test cases and then gave these cases to our tester. We decided to change the number of customer and priority list values to test the performance under the customer number and priority changes.

We used our performance testing results to compare our queuing application's performance under two situations: when the system works without any priority and when the system works with different priorities, also with different customer numbers. We compare these by referring to the total time in simulation versus total customer number in simulation graphs to see if the test case with different priorities takes more time than the test case without any priority. Furthermore, if it does, we observe how much more time it takes to generate simulation than the other test case.

We conducted four test cases:

1. We run our simulation code with arrival rate with 3, service rate with 4, customer number with 1160 ,and with the same priorities.
2. We run our simulation code with arrival rate with 3, service rate with 4, customer number with 1160, and with different priorities (1,2).
3. We run our simulation code with arrival rate with 3, service rate with 4, customer number with 2320, and with different priorities (1,2).
4. We run our simulation code with arrival rate with 3, service rate with 4, customer number with 1160, and with different priorities (1,2,3).

We compared 1 and 2 to analyze our performance with the same number of customers and with different priorities. We examine each total time in simulation versus total customer number in the simulation graph.

We compared 2 and 3 to analyze our performance with the different number of customers and with the same priorities. We examine each total time in simulation versus total customer number in the simulation graph.

We compared 2 and 4 to analyze our performance with the same number of customers and with different priorities. We examine each total time in simulation versus total customer number in the simulation graph.

Our tester for the Performance testing was Eylül.

Our script for the performance testing is the following:

```
233
234         bankSimulation.calculate_metrics()
235         # Plot the graph
236         plt.plot(timer_total_customers, timer_total_time)
237         plt.xlabel('Total Customer Number')
238         plt.ylabel('Total Time (seconds)')
239         plt.title('Graph of Total Time vs Total Customer Number')
240         plt.show()
241         print(timer_total_customers)
242         print(timer_total_time)
243
244
245 ▶   if __name__ == "__main__":
246         #simulation(simulation_customer_number, arrival_rate, service_rate, num_servers, priority_list)
247         simulation(1160, 3, 4, 1, [1]*386+[2]*386+[3]*388)
248
```

*Figure 33: Performance testing script*

We entered different parameters inside the simulation() function call shown in line 247. simulation() function starts the simulation and computes all the needed parameters. Then, it calls the calculate_metrics() function and prints the calculated parameters.

All the needed calculations are done inside the bankSimulation.calculate_metrics() call which is shown below:

```
def calculate_metrics(self):

    print(f"Total customers: {self.total_served_customers}")
    print(f"Total service time: {self.total_service_time}")
    print(f"Total wait time: {self.total_wait_time}")
    print(f"Simulation Time: {self.simulation_time}")
    print(f"Total exit time: {self.total_exit_time}")
```

*Figure 34: Performance testing script continue*

6.4.2 Test cases

| *ID* | *Description* | *Steps* | *Test Data* | *Pre-condition* | *Expected Output* | *Passed/ Failed* |
|---|---|---|---|---|---|---|
| 1 | Check MM1 simulation model performance based on total customer number with same priority | 1.Open test code<br>2.Enter simulation_customer_number, arrival_rate, service_rate,num_servers,priority_list values<br>3.Run the code | simulation_customer_number=1160<br>arrival_rate=3<br>service_rate=4<br>num_servers=1<br>priority_list=[1]*1160 | | shorter simulation system time compared to ID2 test result graph | Passed |
| 2 | Check MM1 simulation model performance based on total customer number with different priority | 1.Open test code<br>2.Enter simulation_customer_number, arrival_rate, service_rate,num_servers,priority_list values<br>3.Run the code | simulation_customer_number=1160<br>arrival_rate=3<br>service_rate=4<br>num_servers=1<br>priority_list=[1]*580 +[2]*580 | | longer simulation system time compared to test ID1 result | Passed |
| 3 | Check MM1 simulation model | 1.Open test code | simulation_customer_number=2320<br>arrival_rate=3 | | longer simulation system time | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | performance based on total customer number with different priority | 2.Enter simulation _customer _number, arrival_rate , service_rat e,num_ser vers,priorit y_list values 3.Run the code | service_rate=4 num_servers=1 priority_list=[1]*580 +[2]*580 | | compared to ID2 test result graph | |
| 4 | Check MM1 simulation model performance based on total customer number with different priority | 1.Open test code 2.Enter simulation _customer _number, arrival_rate , service_rat e,num_ser vers,priorit y_list values 3.Run the code | simulation_customer _number=1160 arrival_rate=3 service_rate=4 num_servers=1 priority_list=[1]*386 +[2]*386+[3]*388 | | shorter simulation system time compared to ID2 test result graph | Passed |

*Table 5: Test case table for Performance testing*

6.4.3 Test Results

As a result, our test script plots the following graphs with total simulation time and total customer number parameters.

test ID1:

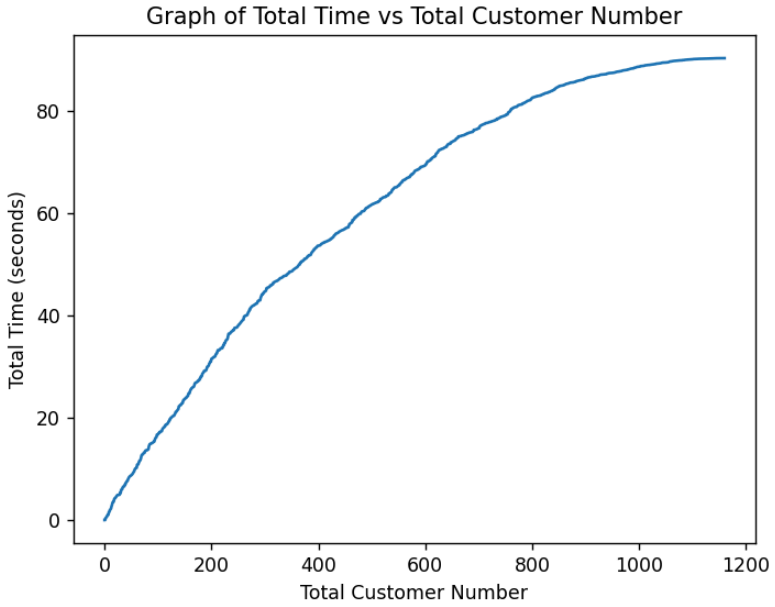



Figure 35: Test result graph for test ID 1

test ID 2:

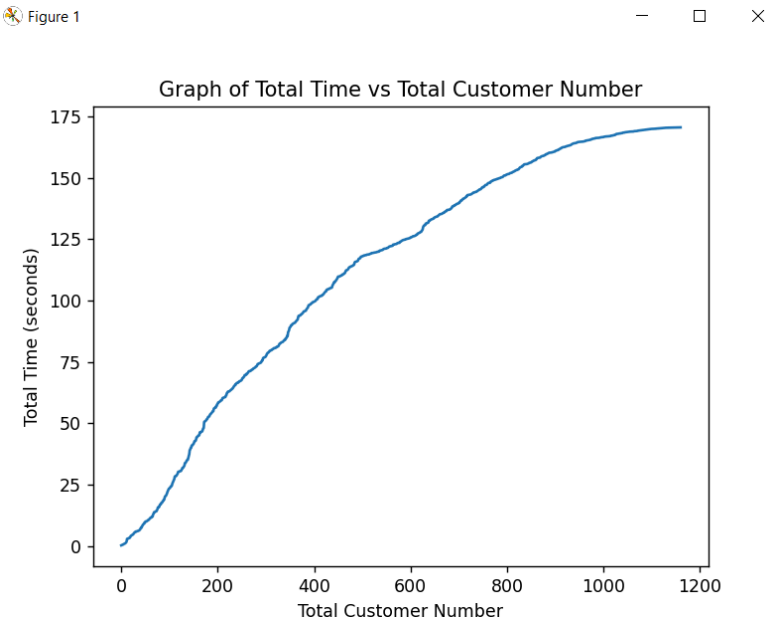



Figure 36: Test result graph for test ID 2

test ID 3:

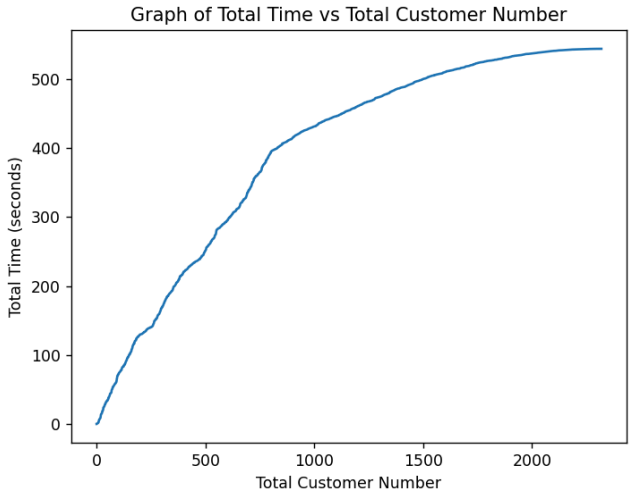



*Figure 37: Test result graph for test ID 3*

test ID 4:


*Figure 38: Test result graph for test ID 4*

### *6.4.4 Test Log*

In this script, we entered 2 different priority values to the "priority_list" parameter. Therefore, many calculations have changed since the system controls the priority of each customer and does position changes when needed. We provide test logs for test ID 3 and test ID4 below.

```
Total customers: 2320
Total service time: 559.2651742668156
Total wait time: 1503.0428673356625
Average service time: 0.24106257511500673
Simulation Time: 777.4324999850721
Total exit time: 2062.8843673248375
```

*Figure 39: Test Log for test ID 3*

```
Total customers: 1160
Total service time: 298.0753323016458
Total wait time: 991.3101467758612
Average service time: 0.2569614933634878
Simulation Time: 387.90899999428314
Total exit time: 1289.675646771441
```

*Figure 40: Test Log for test ID 4*

## 6.5 Usability Testing

### 6.5.1 Test Procedure

We used SUS as a technique to conduct a Usability test. We first created the SUS tests by checking the SUS test examples. These tests include the ratings and statements we want to measure. While deciding on the statements, we focused on reliability, security, and robustness. We conducted the tests on our laptops and stored them there. For the admin side tests, we opened the admin panel on our laptops, and explained the test scenarios to our end-users. After they got familiar with our product, we wanted them to fill the desired tests on our laptops. We repeated these steps for each end-user.

For the employee and customer side tests, we opened our mobile application which is already installed on our mobile devices. We explained the test scenarios to our end-users and gave our mobile devices to them. After they got familiar with our product, we wanted them to fill the desired tests on our laptops. We repeated these steps for each end-user.

By looking at our collected scores, we highlighted our software product's missing parts.

We specified the Usability tests we are planning to in Test Plan report which are the followings:

1. Serving the customer in employee side
2. Entering the queue in customer side
3. Leaving the queue in customer side
4. Assigning employees in admin side
5. Editing customer details in admin side

Our testers for the Performance testing were Eylül and İrem.

### 6.5.2 Expectations

We expect a score between 70-100 which stands for good usability for the mobile application part. Since there is no such a challenging operation, we believe that we provide an user-friendly mobile application.

We expect a score between 80-100 which stands for excellent usability for the admin panel part. We expect higher scores as we made it quite simple to use to assist the admin in handling his/her tasks in an easier way.

We expected different score ranges for mobile application based tests and admin panel based tests as we believed that our admin panel is easier to use when compared to our mobile application. Some end-users can find our mobile application a bit hard to use. This is mainly because users have different backgrounds, experiences. For the ones who have not used any bank related mobile application before may find our application not very easy to use. Even though the application sends the needed alerts, monitoring the active queue screen is the customer's responsibility. Therefore, customers should be capable of using the mobile application. We expected getting different scores from the SUS tests conducted for the mobile application due to the user-based differences.

### 6.5.3 Test Results

We conducted the SUS tests with 5 different users. We got the following results:

1. Entering the queue: 100, 100, 100, 97.5, 97.5, 95
2. Assigning employees: 100, 92.5, 97.5, 95, 100
3. Serving the customer: 95, 97.5, 85, 80, 80
4. Leaving the queue: 97.5, 97.5, 90, 85, 85
5. Editing customer details: 100, 87.5, 100, 95, 95

Even though we expected scores between 70-100 for the mobile application, results were much higher than our expectation.

We got the scores between 80-100 for the admin panel testing as we expected.

### *6.5.4 Test Log*

| | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| 1 | **Editing customer details** | Strongly Disagree | Disagree | Neutral | Agree | Strongly Agree | |
| 2 | | 1 | 2 | 3 | 4 | 5 | |
| 3 | I think that I would like to use this website frequently. | | | | | ✓ | 4*(2.5)=10 |
| 4 | I found the website unnecessarily complex. | ✓ | | | | | 4*(2.5)=10 |
| 5 | I thought the website was easy to use. | | | | | ✓ | 4*(2.5)=10 |
| 6 | I think that I would need the support of a technical person to be able to use this website. | | | | ✓ | | 3*(2.5)=7.5 |
| 7 | I found the various functions in this website were well integrated. | | | | | ✓ | 4*(2.5)=10 |
| 8 | I thought there was too much inconsistency in this system. | ✓ | | | | | 4*(2.5)=10 |
| 9 | I would imagine that most people would learn to use this system very quickly. | | | | ✓ | | 3*(2.5)=7.5 |
| 10 | I found the system very cumbersome to use. | | | ✓ | | | 2*(2.5)=5 |
| 11 | I felt very confident using the system. | | | | | ✓ | 4*(2.5)=10 |
| 12 | I needed to learn a lot of things before I could get going with this system. | | ✓ | | | | 3*(2.5)=7.5 |
| 13 | | | | | | | 87.5 |

*Figure 41: SUS test log example for Entering the queue case*

## 7 Project Scheduling

### 7.1 Milestones and Tasks

Our milestones and tasks are listed below.

**M1 :** Database Implementation

T1 Database Creation in Google Firebase

**M2:** Interface Control

T2 See list of branches

T3 See the state of the queue and estimated waiting time

**M3:** User Control

T4 Registration and Login

**M4:** Queue Handling

T5 Join the queue

T6 Leave the queue

**M5:** Simulation

T7 M/M/1 Code

T8 M/M/C Code

**M6:** Admin Panel

T9: Editing Customers

T10: Editing Queue

T11: Integration with simulation

**M7:** App Improvements

T12: Dynamic Priority

T13: Notification

T14: Taking Statistics

**M8:** Testing

T15: Testing Mobile Application

T16: Testing Simulation

T17: Testing Admin Panel

| Task ID | Task Description | Dependencies | Estimated | Team Members |
|---|---|---|---|---|
| T1 | Database Creation in Google Firebase | | 9 days | Barış |
| T2 | See list of branches | T1(M1) | 7 days | Eylül |
| T3 | See the state of the queue and estimated waiting time | T2 | 9 days | İrem |
| T4 | Registration and Login | T1(M1) | 7 days | Seray |
| T5 | Join the queue | T3(M2) | 14 days | Barış,İrem |
| T6 | Leave the queue | T5 | 10 days | Barış,İrem |
| T7 | M/M/1 Code | T1(M1) | 14 days | Seray,Eylül |
| T8 | M/M/C Code | T7 | 21 days | Seray |
| T9 | Editing Customers | T1(M1) | 5 days | Eylül,İrem |
| T10 | Editing Queue | T5 | 5 days | İrem |
| T11 | Integration with simulation | T8(M5) | 14 days | Eylül |
| T12 | Dynamic Priority | T5 | 21 days | Barış |
| T13 | Notification | T5 | 7 days | Barış |
| T14 | Taking Statistics | T6(M4) | 5 days | Barış |
| T15 | Testing Mobile Application | T6(M4) | 5 days | Eylül,İrem |
| T16 | Testing Simulation | T11(M6),T14(M7) | 3 days | Eylül,İrem |
| T17 | Testing Admin Panel | T10 | 3 days | Eylül,İrem |

*Table 6: Task Table*

## 7.2 Gantt Chart

We drew the first and second semester together. However, we separately put them below for easy reading.

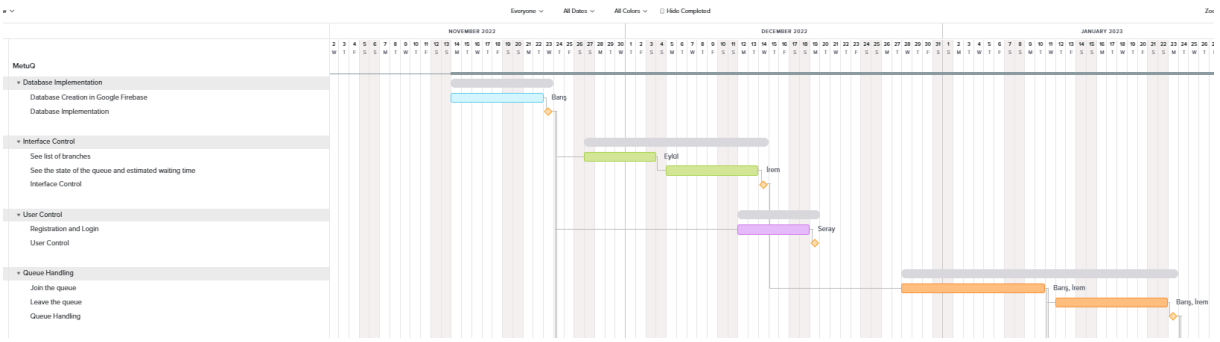



*Figure 42: Gantt Chart for 1st semester*

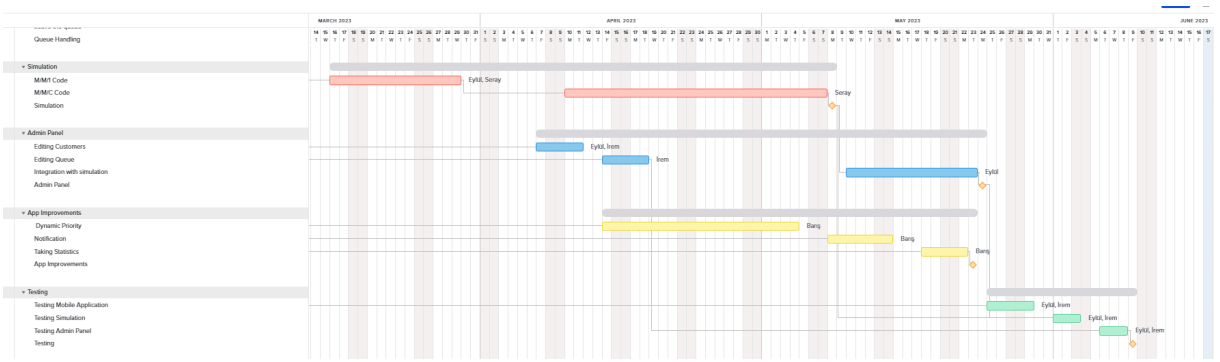



*Figure 43: Gantt Chart for 2nd semester*

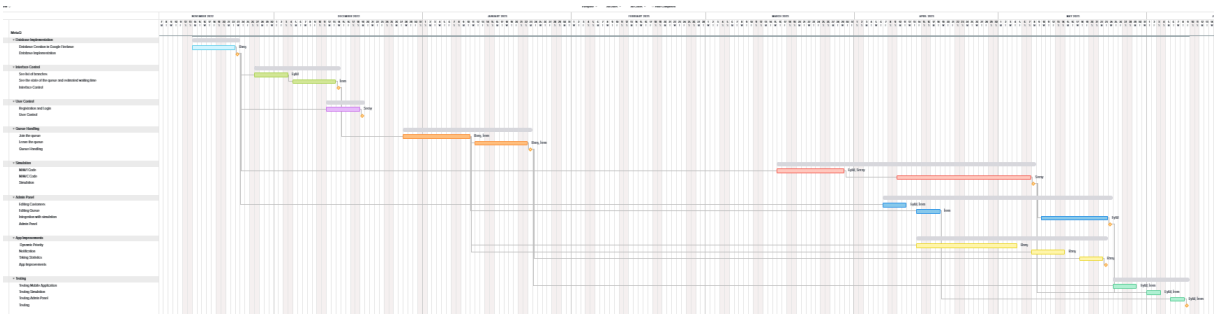


*Figure 44: Gantt Chart for both semester*

# 8 Conclusion

## 8.1 Critical Evaluation of Second Semester

We can concentrate on a few key issues and analyze each of them as follows in order to critically assess our work from the second semester.

We scheduled a few meetings to discuss the implementation plans for the admin panel and simulation, and we discussed about the benefits and drawbacks of several programming languages. We also talked about the features that would be added to the mobile app. After these discussions, we ultimately chose Python for the simulation and Flask for the admin panel for the website. There were no problems throughout the semester because everyone on the team supported it. Each team member could use these languages and work environments in an easy and feasible way because we used them for our projects from the previous semester. We can conclude that the fact that we were able to come to a consensus before starting the implementation was a good start.

A long meeting between the team and our supervisor was convened to discuss the project's planning process. We talked about the constraints of our project and what was expected of us in terms of the final product during this meeting. We made a list of the remaining features of our project and chose to assign one team member to the simulation, two to the web admin panel, and one to the Android application. We always helped each other out while dealing with the implementation of this strategy, and it worked out well. During the brief meetings we had with our supervisor during the semester, she expressed her pleasure with the project's advancement.

We took the workload of our lectures into consideration while managing our time. We had different classes, so our timetables were very different from one another. In order to schedule time for meetings, we have established common days at the start of the semester. Even though it may have seemed like each of us worked on separate sections of the project, they were all interconnected, so we maintained constant communication. As a result, we worked in an organized and systematic manner to complete our project, which consisted of three main components: a mobile application, a simulation, and a web panel, without disrupting the work of any other group members.

We made an effort to get the development process started right away. We integrated a dynamic priority implementation, a feature for customer notifications, and a list of nearby branches into our project on the application side. The simulation part was a completely new addition to this project. The simulation was intended to be used to demonstrate how queues functioned in real-world situations with varying circumstances. Following discussions with the advisor and other instructors during the semester, we made the decision to incorporate simulation into the project for a different and more beneficial reason. Even though this

alteration in the development process resulted in variations in the time estimates we made, we eventually included the simulation in our project. We linked it to the project database and ran a simulation using actual data. As a result of everything said above, the simulation we've included in our software has been developed to the point where it can now advise users about how effectively and logically queue systems perform. In other words, the information users who will utilize the project obtained from the simulation will help them construct queue systems more effectively. In the web panel, we aimed to show the simulation results to the admin and also worked on some features like the admin can change customers' priority, add employees, etc. by using the web panel. Overall, we did a great job this semester, and after talking to our instructors, we added a few extra details to our project.

Regarding the final product, our project's progress was finished in the third sprint after we resolved our challenges with our new features added, time scheduling, and software flaws. We were successful in completing all of the responsibilities as we planned with our supervisor.

In conclusion, while looking at the semester as a whole, our team is happy with the outcome and the work we have completed thus far. We continued to work on our project with the same work ethic and dedication during this time, which allowed us to successfully finish the project we worked on for the entire year.

## 8.2 Retrospective of Second Semester

What succeeded?

We made a really good decision on the project's tasks and necessary components with our supervisor at the start of the semester. As a result, we decided on the functionality, and everyone was aware of the duties they had to perform and the directions they needed to take. We performed the essential additions and improvements since we could clearly picture the application's final version. Establishing the project boundaries at the start of the term helped the project advance. Additionally, as we planned appropriate time slots for the project, we organized many Zoom sessions and face-to-face meetings in order to accomplish our goals. Regarding the scheduling of meetings with our supervisor and with the rest of the team, we had no issues. We also got along really fine helping each other out with the coding issues we were having. Additionally, the addition of extra features to our project had no impact on our time management due to the clean and object-oriented code that we had written. These functionalities were easily implemented by making small adjustments.

What went less well?

We had a problem with feedback for the simulation code. While a simulation code with a different implementation was written until mid-term, the operation of the simulation code was changed with the late feedback and ideas. To avoid this problem, we could have examined the code carefully and in detail and given each other faster and earlier feedback. Although there were some setbacks on the simulation side, we finished it by the third sprint.

What should we do?

We can make improvements to the web and application interfaces for the application we make. As discussed in the Future Work section, we can show the branches on the map via the Google

Map platform to make our mobile application more user-friendly, or we can make the application compatible with different platforms to reach large users.

### 8.3 Future Work Planned for our Project

Here we wanted to make a few comments on how the application could be expanded and what else could be done to make it more user-friendly.

1. Google Map Platform: During the implementation phase of this period, the focus was primarily on improving the functional aspects of the implementation. For this reason, a list of branches from the closest to the furthest was shown to the customers for convenience. For a better user interface and attractiveness, the branches in this list can then also be displayed on the map using the Google Map platform.
2. Different Platforms: Currently, the application can only be used for android users, so it can be considered to make an application for IOS users. A similarly simple web application can also be used for non-mobile users.

## 9 References

[1] Virtual queuing system by Wavetec. Wavetec. (2022, November 2). Retrieved from

https://www.wavetec.com/solutions/queue-management/virtual-queuing/

[2] Market-leading walk-in Virtual Queue system: For enterprise. Retail Choreography

Software. (2023, January 6). Retrieved January 10, 2023, from

https://www.qudini.com/solutions/queue-management-system.

[3] Virtual Smart Queue Management System: 2Meters App. 2Meters. (2022, November 29).

Retrieved January 10, 2023, from https://2meters.app/

## 10 Appendices

### 10.1 Acronyms and abbreviations

DB: Database

ER : Entity Relationship

UTFP: Compute UnadjustedTotal of Function Points

LOC: Compute Estimated Lines of Code

VAF:ValueAdjustment Factor

Auth:Authentication

ATFP:AdjustedTotal of Function Points

KDSI: Kilo Delivered Source Instruction

TDEV:The development time

MM: Effort in Man-Months

MQL : Mean Queue Length

### 10.2 Glossary

- MM1:

MM1 represents a specific type of queuing system with the following characteristics:

M: Markovian arrivals: The arrival of customers follows a Poisson process, which means the time between consecutive arrivals is exponentially distributed.

M: Markovian service time: The service times are exponentially distributed, and each customer is served independently of others.

1: Single server: There is only one server in the system.

- MMC:

MMC is another type of queuing system with the following characteristics:

M: Markovian arrivals: The arrival of customers follows a Poisson process.

M: Markovian service time: The service times are exponentially distributed.

c: Multiple servers: There are c servers available for serving customers simultaneously.

- Rho (ρ) :

Rho is a performance metric used in queuing theory to represent the traffic intensity or utilization of a queuing system. It is defined as the ratio of the average arrival rate (λ) to the average service rate (μ)

- MQL (Mean Queue Length):

It is a performance metric used to measure the average number of customers in the queue.